

Human review packet: GN21DriverSurgePricing

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GN21DriverSurgePricing

Paper id: GN21DriverSurgePricing

Source version: Management Science 2022 / arXiv v4 (2021); byte-pinned publication source record

Source record: audit/paper_statement_map.json

Rows: 18

Generated: 2026-08-19

Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GN21DriverSurgePricing --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

Contents

- [Semantic library prerequisites \(3; not paper claims\)](#)
 - [EconCSLib.positiveRealAcceptAll](#)
 - [EconCSLib.acceptsAllPositiveReal](#)
 - [EconCSLib.twoStateCtmcSwitchProb](#)
- [Source-model definitions \(1\)](#)
 - [review_definition_surge_stateSpec](#)
- [Appendix source claims \(1\)](#)
 - [review_lemma10_nonsurge_derivative_positive_of_acceptAll_boundsSpec](#)
- [Main-text source claims \(3\)](#)
 - [review_lemma1_measured_dynamic_reward_decompositionSpec](#)
 - [review_lemma2_switch_probability_formulaSpec](#)
 - [review_lemma3_measured_time_fraction_formulaSpec](#)
- [Appendix source claims \(continued\) \(5\)](#)
 - [review_lemma4_single_state_threshold_uniquenessSpec](#)
 - [review_lemma6_upper_endpoint_derivative_formulaSpec](#)
 - [review_lemma7_affine_positive_additive_response_quasi_convexSpec](#)
 - [review_lemma8_affine_negative_additive_response_quasi_concaveSpec](#)
 - [review_lemma9_surge_derivative_positive_of_acceptAll_boundsSpec](#)
- [Main-text source claims \(continued\) \(3\)](#)
 - [review_proposition3_1_affine_single_state_icSpec](#)
 - [review_theorem1_single_state_threshold_best_responseSpec](#)
 - [review_theorem2_multiplicative_positive_finite_cutoff_not_ic_both_statesSpec](#)
- [Appendix source claims \(continued 2\) \(1\)](#)
 - [review_theorem4_full_structural_policy_forms_directSpec](#)
- [Main-text source claims \(continued 2\) \(1\)](#)
 - [review_theorem2_multiplicative_policy_shape_source_claimSpec](#)

- [Appendix source claims \(continued 3\) \(1\)](#)
 - [lemma5_full_variational_policy_formsSpec](#)
- [Source-model definitions \(continued\) \(1\)](#)
 - [review_definition_incentive_compatibleSpec](#)
- [Main-text source claims \(continued 3\) \(1\)](#)
 - [review_theorem3_structured_pricingSpec](#)

Material library prerequisites

EconCSLib.positiveRealAcceptAll

Source locator: cited publication, lines 264-287

Verbatim paper-source connection

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure θ .

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is $R(w, \sigma), \liminf_{t \rightarrow \infty}$

$$\frac{R(w, \sigma, t)}{t}$$

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives. A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w . In other words, payment function w is incentive compatible if an earnings-maximizing driver (who knows all the primitives, w , and the trip length τ at request time) accepts every trip request.

Lean-expanded library semantic target

$\forall (a : \mathbb{R}), \text{Set.Ioi } 0 \ a$

Exact Lean library declaration

abbrev positiveRealAcceptAll : Set $\mathbb{R}$:=
Set.Ioi 0

Recorded source-to-library screening Verdict: matches

Reason: The source names the accept-all policy as the positive trip-length domain $(0, \infty)$. This reusable set declaration is exactly that domain, with no additional cutoff or endpoint convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.acceptsAllPositiveReal

Source locator: cited publication, lines 264-287

Verbatim paper-source connection

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is $R(w, \sigma), \liminf_{t \rightarrow \infty}$

$R(w, \sigma, t)$
.
 t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (\emptyset, \infty)$ in the single-state model or $\sigma = \{(\emptyset, \infty), (\emptyset, \infty)\}$ in the dynamic model is optimal with respect to w . In other words, payment function w is incentive compatible if an earnings-maximizing driver (who knows all the primitives, w , and the trip length τ at request time) accepts every trip request.

Lean-expanded library semantic target

$\forall (\sigma : \text{Set } \mathbb{R}), \text{EconCSLib.positiveRealAcceptAll} \subseteq \sigma$

Exact Lean library declaration

abbrev acceptsAllPositiveReal ($\sigma : \text{Set } \mathbb{R}$) : Prop :=
positiveRealAcceptAll $\subseteq \sigma$

Recorded source-to-library screening Verdict: matches

Reason: The source's accept-all policy accepts every positive trip request. The Lean predicate is precisely membership of every positive trip length in that policy.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.twoStateCtmcSwitchProb

Source locator: cited publication, lines 657-670

Verbatim paper-source connection

Lemma 2. Suppose the world is in state i at time t . Let $q_{i \rightarrow j}(s)$ denote the probability that the world will be in state $j \neq i$ at time $t + s$. Then,

$$q_{i \rightarrow j}(s) = \frac{\lambda_{i \rightarrow j}}{\lambda_{i \rightarrow j} + \lambda_{j \rightarrow i}} (1 - e^{-(\lambda_{i \rightarrow j} + \lambda_{j \rightarrow i})s})$$

Note that $q_{i \rightarrow j}(s)$ is not just the probability that the world state transitions once during time $(t, t + s)$, but the probability that it transitions an odd number of times. This formulation emerges through a standard analysis of two-state CTMCs, in which this probability can be found through the inverse of the Laplace transform of the inverse of the resolvent of the Q -matrix for the system. Incorporating this value in closed form is the main hurdle in extending our results to general systems with more than two states. Using this formulation, the following lemma shows $\mu_i(\sigma)$.

Lean-expanded library semantic target

```
(lambdaIJ lambdaJI t : ℝ) → lambdaIJ / (lambdaIJ + lambdaJI) * (1 - Real.exp (-(lambdaIJ + lambdaJI) * t))
```

Exact Lean library declaration

```
noncomputable def twoStateCtmcSwitchProb (lambdaIJ lambdaJI t : ℝ) : ℝ :=  
  lambdaIJ / (lambdaIJ + lambdaJI) *  
  (1 - Real.exp (-(lambdaIJ + lambdaJI) * t))
```

Recorded source-to-library screening Verdict: matches

Reason: Lemma 2 gives the two-state continuous-time Markov-chain switching probability as the stated exponential-rate expression. The reusable Lean declaration is that same probability function, with the paper-local Spec supplying its positivity domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source-model definitions

Review row 1

Source locator: cited publication, lines 377-381

Verbatim source input

Finally, we can now precisely define what it means for $i = 2$ to be the surge state: it has a higher potential earning rate than state 1. There exists some policy σ_2 such that $R_2(w_2, \sigma_2) > R_1(w_1, \sigma_1)$, for all $\sigma_1 \in R^+$. In other words, suppose that instead we were in the single-state setting, where the primitives were set as either (λ_1, F_1, w_1) or (λ_2, F_2, w_2) . Then the latter set of primitives would yield a higher maximum earnings.⁵

Expanded PaperInterface specification

```
∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ)
  (w : Fin 2 → GN21DriverSurgePricing.PricingFunction),
  GN21DriverSurgePricing.gn21SourceSurgeStateDominance mu arrival w
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_definition_surge_state_realizes_spec

Recorded source-to-Spec screening Verdict: matches

Reason: The source defines state 2 as a surge state when one state-2 policy has a reward rate strictly above every feasible state-1 policy. The transparent Spec preserves that existential policy, universal comparison, strict inequality, and open-policy domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix source claims

Review row 2

Source locator: cited publication, lines 3828-3829

Verbatim source input

Lemma 10. Fix arbitrary σ_2 , and thus Q_2 , T_2 , R_2 . Let $\bar{Q}_1$, $\bar{T}_1$ be the respective values of Q_1 , T_1 at $\sigma_1 = (0, \infty)$. Let $w_1(\tau) = m\tau + zq_{1 \rightarrow 2}(\tau)$, where $m = R_2$.

If
-
$$\frac{z}{1} (T_2 \lambda_{1 \rightarrow 2} + Q_2) < R_2 Q_2 (\lambda_{1 \rightarrow 2} \bar{T}_1 - \bar{Q}_1) + \lambda_{1 \rightarrow 2} (T_2 \lambda_{1 \rightarrow 2} + Q_2) (\bar{Q}_1 - \lambda_{1 \rightarrow 2})$$

Then ∂u
 $R(w, \sigma) > 0$, for all u , σ_1 . Furthermore, the constraint set is feasible regardless of the primitives.

D.4

[Next verbatim source excerpt]

We can now do the same thing for the first state, assuming that $w_1(\tau)$ is of the form $w_1(\tau) = m\tau + zq_{1 \rightarrow 2}(\tau)$, where now $z \leq 0$ and $m = R_2$. In the next lemma, we consider u an upper endpoint

[Next verbatim source excerpt]

[form-feed in source extraction]

Proof. Proof. Similar to previous proof. Suppose we have $w_1(u) = mu + zq_{1 \rightarrow 2}(u)$, for some $m = R_2$, $z \leq 0$.

Expanded PaperInterface specification

```

V (arrivalRate lowerEndpoint u T2 Q2 T1 Q1 Tbar1 Qbar1 switch12 switch21 R2 z ratio : ℝ) (density : ℝ → ℝ)
(harrival_pos : 0 < arrivalRate) (hdensity_pos : 0 < density u) (hQ2_pos : 0 < Q2)
(hden :
  GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch12 lowerEndpoint density
    (GN21DriverSurgePricing.gn21SwitchProb switch12 switch21) u *
    T2 +
    Q2 * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u ≠
    0)
(hq_int :
  IntervalIntegrable (fun τ => GN21DriverSurgePricing.gn21SwitchProb switch12 switch21 τ * density τ)
    MeasureTheory.volume lowerEndpoint u)
(hq_meas :
  StronglyMeasurableAtFilter (fun τ => GN21DriverSurgePricing.gn21SwitchProb switch12 switch21 τ * density τ) (nhds u)
    MeasureTheory.volume)
(hw_int :
  IntervalIntegrable (fun τ => GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 τ * density τ)
    MeasureTheory.volume lowerEndpoint u)
(hw_meas :
  StronglyMeasurableAtFilter
    (fun τ => GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 τ * density τ) (nhds u)
    MeasureTheory.volume)
(hw_cont :
  ContinuousAt (fun τ => GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 τ * density τ) u)
(ht_int : IntervalIntegrable (fun τ => τ * density τ) MeasureTheory.volume lowerEndpoint u)
(ht_meas : StronglyMeasurableAtFilter (fun τ => τ * density τ) (nhds u) MeasureTheory.volume)
(ht_cont : ContinuousAt (fun τ => τ * density τ) u),
GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch12 lowerEndpoint density
  (GN21DriverSurgePricing.gn21SwitchProb switch12 switch21) u =
  Q1 →
  GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u = T1 →
  GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density
    (GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21) u =
  R2 * (T1 - 1) + z * (Q1 - switch12) →
  GN21DriverSurgePricing.lemma10StructuredBounds ratio T2 Q2 Tbar1 Qbar1 switch12 →
  z ≤ 0 →
  z = ratio * R2 →
  0 < R2 →
  0 < switch12 →
  0 < switch12 + switch21 →
  0 < u →
  0 < T2 * switch12 + Q2 →
  0 ≤ switch12 * T1 - Q1 →
  switch12 * T1 - Q1 ≤ switch12 * Tbar1 - Qbar1 →
  switch12 < Q1 →
  Q1 ≤ Qbar1 →
  0 <
    (GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths
      arrivalRate switch12 lowerEndpoint u Q2 T2 (R2 * T2) density
      (GN21DriverSurgePricing.gn21SwitchProb switch12 switch21))

```

```
(GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21)
harrival_pos hdensity_pos hQ2_pos hden hq_int hq_meas hq_cont hw_int hw_meas
hw_cont ht_int ht_meas ht_cont).derivativeValue
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma10_nonsurge_derivative_positive_of_acceptAll_bounds

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 10 fixes the surge policy, uses the displayed non-surge price and accept-all quantities, and concludes positivity of the relevant non-surge upper-endpoint derivative. The expanded Spec makes the source inequality domain and derivative statement explicit.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims

Review row 3

Source locator: cited publication, lines 324-367

Verbatim source input

Lemma 1. In the dynamic model, the earnings rate can be decomposed into each state i earnings rate $R_i(w_i, \sigma_i)$ and fraction of time $\mu_i(\sigma)$ spent in state i :

$$R(w, \sigma) = \mu_1(\sigma)R_1(w_1, \sigma_1) + \mu_2(\sigma)R_2(w_2, \sigma_2)$$

with probability 1.

$i(\sigma_i)$
As in the single-state model, $R_i(w_i, \sigma_i) = W$
 $T_i(\sigma_i)$, where

$$\begin{aligned} & \sum_{i=1}^2 W_i(\sigma_i) = \\ & w_i(\tau) dF_i(\tau), \\ & F_i(\sigma_i) \tau \in \sigma_i \\ & \sum_{i=1}^2 T_i(\sigma_i) = \\ & + \lambda_i F_i(\sigma_i) F_i(\sigma_i) \\ & \sum_{i=1}^2 \tau dF_i(\tau) \\ & \tau \in \sigma_i \end{aligned}$$

We prove the result by defining a new renewal process, in which a single reward renewal cycle is: the time between the driver is open in state 1 to the next time the driver is open in state 1 after being open in state 2 at least once. In other words, each renewal cycle is composed of some number (potentially zero) of sub-cycles in which the driver is open in state 1 and then is open in state 1 again after a completed trip; one sub-cycle starting with the driver open in state 1 and ending with being open in state 2 (either after a completed trip or a state transition while open); some number (potentially zero) of sub-cycles in which the driver is open in state 2 and then is open in state 2 again after a completed trip; and finally one sub-cycle starting in state 2 and ending with the driver open in state 1.

Given the number of such renewal reward cycles completed up to time t , the total earnings on trips starting in each state (earnings in each sub-cycle) are independent of each other, and then we use Wald's identity (Wald, 1973) to separate $\mu_i(\sigma)$ and $R_i(\sigma_i)$.

Note that $T_i(\sigma_i)$ is not exactly the expected length of time in a single sub-cycle in a state given σ_i , but rather is proportional to it; the multiplicative constant $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$ cancels out with the same constant in the expected earnings in a single sub-cycle in a state given σ_i . This constant emerges from the primitives: when the driver is open in state i , there are two competing exponential clocks (with rates $\lambda_i F_i(\sigma_i)$ and $\lambda_{i \rightarrow j}$, respectively) that determine whether the driver will accept a request before the world state changes.

Expanded PaperInterface specification

```
∀ {Omega : Type u_1} [inst : MeasurableSpace Omega] {POmega : MeasureTheory.Measure Omega}
{muI muJ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrivalI arrivalJ switchIJ switchJI : ℝ}
{wI wJ : GN21DriverSurgePricing.PricingFunction} {sigmaI sigmaJ : GN21DriverSurgePricing.TripPolicy}
(C :
  GN21DriverSurgePricing.GN21DynamicIIDCycleModel POmega muI muJ arrivalI arrivalJ switchIJ switchJI wI wJ sigmaI
  sigmaJ),
∀ (omega : Omega) @POmega,
Filter.Tendsto
  (fun n =>
    (∑ k ∈ Finset.range n, C.stateEarningI k omega + ∑ k ∈ Finset.range n, C.stateEarningJ k omega) /
    (∑ k ∈ Finset.range n, C.stateTimeI k omega + ∑ k ∈ Finset.range n, C.stateTimeJ k omega))
  Filter.atTop
  (nhds
    (GN21DriverSurgePricing.gn21MeasuredDynamicReward muI muJ arrivalI arrivalJ switchIJ switchJI wI wJ sigmaI
    sigmaJ))
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemmal_measured_dynamic_reward_decomposition

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 1 states the almost-sure decomposition of dynamic earnings into state-time fractions times state earnings rates. The transparent Spec is the renewal-cycle almost-sure convergence statement with the displayed weighted reward formula and its explicit IID-cycle model inputs.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 657-670

Verbatim source input

Lemma 2. Suppose the world is in state i at time t . Let $q_{i \rightarrow j}(s)$ denote the probability that the world will be in state $j \neq i$ at time $t + s$. Then,

$$q_{i \rightarrow j}(s) = \frac{\lambda_{i \rightarrow j}}{\lambda_{i \rightarrow j} + \lambda_{j \rightarrow i}} (1 - e^{-(\lambda_{i \rightarrow j} + \lambda_{j \rightarrow i})s})$$

Note that $q_{i \rightarrow j}(s)$ is not just the probability that the world state transitions once during time $(t, t + s)$, but the probability that it transitions an odd number of times. This formulation emerges through a standard analysis of two-state CTMCs, in which this probability can be found through the inverse of the Laplace transform of the inverse of the resolvent of the Q -matrix for the system. Incorporating this value in closed form is the main hurdle in extending our results to general systems with more than two states. Using this formulation, the following lemma shows $\mu_i(\sigma)$.

Expanded PaperInterface specification

```
∀ (lambdaIJ lambdaJI s : ℝ),
  GN21DriverSurgePricing.gn21SwitchProb lambdaIJ lambdaJI s =
    lambdaIJ / (lambdaIJ + lambdaJI) * (1 - Real.exp (-(lambdaIJ + lambdaJI) * s))
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma2_switch_probability_formula

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 2 gives the probability of a state switch during a trip as the displayed CTMC exponential formula. The Spec states that formula for the two switch rates and trip duration, including the positive-rate denominator condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 671-696

Verbatim source input

Lemma 3. Let $T_i(\sigma_i)$ be as defined in Lemma 1. The fraction of time a driver following strategy $\sigma = \{\sigma_1, \sigma_2\}$ spends either open in state i or on a trip started in state i is

$$\lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j) + \lambda_j F_j(\sigma_j) T_j(\sigma_j) Q_i(\sigma_i) + \lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)$$
$$Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$$
$$q_{i \rightarrow j}(\tau) dF_i(\tau)$$
$$\mu_i(\sigma) =$$

where

$\tau \in \sigma_i$

We prove this lemma by finding the expected number of sub-cycles in each state i , i.e., within a larger renewal reward cycle as defined, the expected number of sub-cycles that start with the driver

[form-feed in source extraction]
being open in state i . This expectation is the mean of a geometric random variable parameterized by the probability that the driver will next be open in state j , given the driver is currently open in state i . $Q_i(\sigma_i)$ is proportional to this probability. (As with $T_i(\sigma_i)$, there is a normalizing constant 1
 $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$); the larger it is, the fewer sub-cycles spent in state i . It has two components: the first is the probability that the state changes before the driver accepts a trip request; the second is the probability that the world state is j when the driver completes a trip. Thus, the numerator in $\mu_i(\sigma)$ is proportional to the length of a sub-cycle in state i , times the fraction of sub-cycles that are started in state i . The larger $Q_j(\sigma_j)$ or $T_i(\sigma_i)$, the more time the driver spends in state i .

Expanded PaperInterface specification

```
∀ {Omega : Type u_1} [inst : MeasurableSpace Omega] {POmega : MeasureTheory.Measure Omega}
{muI muJ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrivalI arrivalJ switchIJ switchJI : ℝ}
{sigmaI sigmaJ : GN21DriverSurgePricing.TripPolicy}
(C :
  GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel POmega muI muJ arrivalI arrivalJ switchIJ switchJI sigmaI
  sigmaJ),
∀ (omega : Omega) ∂POmega,
  Filter.Tendsto
    (fun n =>
      (∑ k ∈ Finset.range n, C.stateTimeI k omega) /
      (∑ k ∈ Finset.range n, C.stateTimeI k omega + ∑ k ∈ Finset.range n, C.stateTimeJ k omega))
    Filter.atTop
    (nhds (GN21DriverSurgePricing.gn21MeasuredTimeFraction muI muJ arrivalI arrivalJ switchIJ switchJI sigmaI sigmaJ))
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma3_measured_time_fraction_formula

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 3 identifies the almost-sure fraction of time spent in each state with the displayed expression in T and Q . The expanded Spec keeps the sample-path convergence, both state components, and the renewal-cycle model assumptions explicit.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix source claims (continued)

Review row 6

Source locator: cited publication, lines 2841-2853

Verbatim source input

```
Lemma 4. Consider the single-state model. There exists an optimal policy  $\sigma^*$  of the form  $\sigma^* =$ 
)
*
*
*
{ $\tau : w(\tau$ 
 $\tau \geq c$  } such that  $R(\sigma^*) = c$  . Furthermore, this policy is the unique optimal policy, up to
)
*
sets of measure 0 and up to modifications (subtractions) of sets  $\{\tau : w(\tau$ 
 $\tau = c$  }.
)
 $\geq c^*$  },

[Next verbatim source excerpt]

-10
-20

(699.0, 3598.0]
1
```

Expanded PaperInterface specification

```
∀ (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ)
(w : GN21DriverSurgePricing.PricingFunction),
(Measurable fun τ => w τ / τ) →
(∀ (τ : GN21DriverSurgePricing.TripLength), 0 < τ → 0 ≤ w τ / τ) →
μ GN21DriverSurgePricing.acceptAllPolicy ≠ τ →
MeasureTheory.IntegrableOn w GN21DriverSurgePricing.acceptAllPolicy μ →
MeasureTheory.IntegrableOn (fun τ => τ) GN21DriverSurgePricing.acceptAllPolicy μ →
0 < arrivalRate →
∃ cstar,
0 ≤ cstar ∧
∃ σstar,
GN21DriverSurgePricing.thresholdRatePolicy w cstar σstar ∧
GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w σstar = cstar ∧
GN21DriverSurgePricing.singleStateMeasurableOptimal
(GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w) σstar ∧
∀ ρ ≤ GN21DriverSurgePricing.acceptAllPolicy,
MeasurableSet ρ →
GN21DriverSurgePricing.singleStateMeasurableOptimal
(GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w) ρ →
GN21DriverSurgePricing.singleStateTripMass μ
(GN21DriverSurgePricing.strictThresholdPolicy w cstar \ ρ) =
0 ∧
GN21DriverSurgePricing.singleStateTripMass μ
(ρ \ GN21DriverSurgePricing.completeThresholdPolicy w cstar) =
0
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma4_single_state_threshold_uniqueness

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 4 says that a single-state optimal policy is unique up to changes on a zero-measure set under the strict threshold comparison. The Spec gives the two optimal-policy hypotheses and exactly the source null-set equality conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 3724-3767

Verbatim source input

Lemma 6. Let $R(w, \sigma)$ be as defined in Lemma 1. Then, ∂u
[U+0013 control character]
[U+0012 control character]
[U+0013 control character] [U+0012 control character]

$q_{i \rightarrow j}(u)$
 Q_j
 Q_i
 $w_i(u) Q_i Q_j$
 $r(u, i, w, \sigma)$,
+
-
 $R_j +$
 R_i
 $\Delta j_i +$
 u
 u
 T_i
 T_j
 T_i
 T_j

In other words, $r(u, i, w, \sigma)$ has the same sign as the derivative of the overall reward with respect to u (an upper endpoint of σ_i) at w, σ , but it is not necessarily monotonic with it.

Remark 1. Given assumptions on F_i, w_i :

- $R_i(\sigma), R(\sigma), \mu_i$ are continuous in σ
 - ∂u
- $R(w, \sigma), r(u, i, w, \sigma)$ are both continuous in u (for fixed σ), and continuous in σ .

•
 $q_{i \rightarrow j}(u)$
is strictly decreasing in u .
 u

- If $\Delta j_i < 0$ ($i = 2$ the surge state) and $w_{iu}(u)$ is non-decreasing in u , then $r(u, i, w, \sigma)$ is
 ∂
 $R(w, \sigma)$ is negative up to a certain point
strictly increasing in u for a fixed σ . Thus, ∂u
 $U \in (0, \infty) \cup \{\infty\}$ and then positive thereafter.
- If $\Delta j_i > 0$ ($i = 1$ the non-surge state) and $w_{iu}(u)$ is non-increasing in u , then $r(u, i, w, \sigma)$
 ∂
is strictly decreasing in u for a fixed σ . Thus, ∂u
 $R(w, \sigma)$ is positive up to a certain point

Expanded PaperInterface specification

$\forall$ (arrivalRate switchRate lowerEndpoint $u Q_j T_j W_j R_i R_j : \mathbb{R}$) (density switchProb payment : $\mathbb{R} \rightarrow \mathbb{R}$),
 $0 < \text{arrivalRate} \rightarrow$
 $0 < u \rightarrow$
 $0 < Q_j \rightarrow$
 $0 < \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u \rightarrow$
 $0 < T_j \rightarrow$
 $\text{GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density payment } u =$
 $R_i * \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u \rightarrow$
 $W_j = R_j * T_j \rightarrow$
 $\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb } u *$
 $T_j +$
 $Q_j * \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u \neq$
 $0 \rightarrow$
 IntervalIntegrable (fun $\tau \Rightarrow \text{switchProb } \tau * \text{density } \tau$) MeasureTheory.volume lowerEndpoint $u \rightarrow$
 StronglyMeasurableAtFilter (fun $\tau \Rightarrow \text{switchProb } \tau * \text{density } \tau$) (nhds u) MeasureTheory.volume $\rightarrow$
 ContinuousAt (fun $\tau \Rightarrow \text{switchProb } \tau * \text{density } \tau$) $u \rightarrow$
 IntervalIntegrable (fun $\tau \Rightarrow \text{payment } \tau * \text{density } \tau$) MeasureTheory.volume lowerEndpoint $u \rightarrow$
 StronglyMeasurableAtFilter (fun $\tau \Rightarrow \text{payment } \tau * \text{density } \tau$) (nhds u) MeasureTheory.volume $\rightarrow$
 ContinuousAt (fun $\tau \Rightarrow \text{payment } \tau * \text{density } \tau$) $u \rightarrow$
 IntervalIntegrable (fun $\tau \Rightarrow \tau * \text{density } \tau$) MeasureTheory.volume lowerEndpoint $u \rightarrow$
 StronglyMeasurableAtFilter (fun $\tau \Rightarrow \tau * \text{density } \tau$) (nhds u) MeasureTheory.volume $\rightarrow$
 ContinuousAt (fun $\tau \Rightarrow \tau * \text{density } \tau$) $u \rightarrow$
 HasDerivAt
 (fun $x \Rightarrow$
 $\text{GN21DriverSurgePricing.gn21AggregateDynamicReward}$
 $(\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate}$
 $\text{lowerEndpoint density switchProb } x)$
 Q_j
 $(\text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density}$
 $x)$
 T_j
 $(\text{GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density}$
 $\text{payment } x)$
 $W_j)$
 $(\text{arrivalRate} * \text{density } u * Q_j *$
 $\text{GN21DriverSurgePricing.gn21DerivativeSignKernel (switchProb } u) u (\text{payment } u)$
 $(\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate}$
 $\text{lowerEndpoint density switchProb } u)$
 Q_j
 $(\text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint}$
 $\text{density } u)$

```

      Tj
      (GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint
       density payment u)
    Wj /
    (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate
     lowerEndpoint density switchProb u *
     Tj +
     Qj *
     GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint
     density u) ^
    2)
  u ^
  (0 < density u →
   GN21DriverSurgePricing.sameStrictSign
   (arrivalRate * density u * Qj *
    GN21DriverSurgePricing.gn21DerivativeSignKernel (switchProb u) u
    (payment u)
    (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate
     lowerEndpoint density switchProb u)
    Qj
    (GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint
     density u)
    Tj
    (GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint
     density payment u)
    Wj /
    (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate
     lowerEndpoint density switchProb u *
     Tj +
     Qj *
     GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint
     density u) ^
    2)
   (GN21DriverSurgePricing.gn21Lemma6Response (switchProb u) u (payment u)
    (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate
     lowerEndpoint density switchProb u)
    Qj
    (GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density
     u)
    Tj Ri Rj))

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma6_upper_endpoint_derivative_formula

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 6 gives the upper-endpoint derivative formula for the dynamic reward and uses positive endpoint density only for the strict sign consequence. The expanded Spec retains the formula and separates that conditional sign transfer, matching the selected source statement and proof context.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 3773-3774

Verbatim source input

Lemma 7. Suppose $w_i(\tau) = m\tau + a$, where $m, a > 0$. Then, $r(u, i, w, \sigma)$ is strictly quasi-convex in u , for each fixed σ where $\Delta j_i \leq 0$.

Expanded PaperInterface specification

```
∀ (m a Qi Qj Ti Tj Ri Rj lambdaIJ lambdaJI : ℝ),
  0 < m →
  0 < a →
  Rj - Ri ≤ 0 →
  0 < Qi / Ti + Qj / Tj →
  0 < lambdaIJ →
  0 < lambdaIJ + lambdaJI →
  Ti ≠ 0 →
  Tj ≠ 0 →
  GN21DriverSurgePricing.strictQuasiConvexOnPositive fun u =>
    GN21DriverSurgePricing.gn21Lemma6Response
      (GN21DriverSurgePricing.gn21SwitchProb lambdaIJ lambdaJI u) u (m * u + a) Qi Qj Ti Tj Ri Rj
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma7_affine_positive_additive_response_quasi_convex

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 7 identifies the response under a positive additive affine price as strictly quasi-convex. The Spec gives the same price form, coefficient condition, and quasi-convex response conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 3775-3809

Verbatim source input

Lemma 8. Suppose $w_i(\tau) = m\tau + a$, where $m > 0$ and $a < 0$. Then, $r(u, i, w, \sigma)$ is strictly quasi-concave in u , for each fixed σ where $\Delta_{ji} \geq 0$.

D.3.4

Lemmas for IC policy

Remark 2.

Let

Then

$$w_i(u) = mu + z_{i \rightarrow j}(u)$$

$$w_i = m(T_i - 1) + z(Q_i - \lambda_{i \rightarrow j})$$

$$\frac{\partial}{\partial u} R(w, \sigma) \propto q_{i \rightarrow j}(u) [(R_j - m)T_j T_i + mT_j + zQ_j T_i + zT_j \lambda_{i \rightarrow j}]$$

$$+ u [Q_i T_j (m - R_j) + Q_j (m - zQ_i + z\lambda_{i \rightarrow j})]$$

q

Remark 3. $\lim_{u \rightarrow 0} i \rightarrow j u$

(u)

$$= \lambda_{i \rightarrow j}.$$

Remark 4. $\lambda_{i \rightarrow j} T_i - Q_i \geq 0$ and maximized when $\sigma_i = (0, \infty)$. Similarly, $Q_i \geq 0$ and maximized when $\sigma_i = (0, \infty)$.

$\frac{\partial}{\partial u}$
In the next lemma, we consider u an upper endpoint of σ_2 , and so $\frac{\partial u}{\partial \sigma_1}$

$R(w = \{w_1, w_2\}, \sigma = \{\sigma_1, \sigma_2\})$ is a derivative with respect to an upper endpoint of σ_2 .

57

[form-feed in source extraction]

Lemma 9. Fix arbitrary σ_1 , and thus Q_1, T_1, R_1 . Let $\bar{Q}_2, \bar{T}_2$ be the respective values of Q_2, T_2 at

Expanded PaperInterface specification

```
∀ (m a Q_i Q_j T_i T_j R_i R_j lambda_IJ lambda_JI : ℝ),
  0 < m →
  a < 0 →
  0 ≤ R_j - R_i →
  0 < Q_i / T_i + Q_j / T_j →
  0 < lambda_IJ →
  0 < lambda_IJ + lambda_JI →
  T_i ≠ 0 →
  T_j ≠ 0 →
  GN21DriverSurgePricing.strictQuasiConcaveOnPositive fun u =>
    GN21DriverSurgePricing.gn21Lemma6Response
      (GN21DriverSurgePricing.gn21SwitchProb lambda_IJ lambda_JI u) u (m * u + a) Q_i Q_j T_i T_j R_i R_j
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma8_affine_negative_additive_response_quasi_concave

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 8 identifies the response under the corresponding negative additive affine price as strictly quasi-concave. The Spec preserves the source price form, sign condition, and quasi-concavity conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 704-708

Verbatim source input

Lemma 9. Fix arbitrary σ_1 , and thus Q_1 , T_1 , R_1 . Let $\bar{Q}_2$, $\bar{T}_2$ be the respective values of Q_2 , T_2 at $\sigma_2 = (0, \infty)$. Let $w_2(\tau) = m\tau + zq_{2 \rightarrow 1}(\tau)$, where $m > R_1$.

If
 z
 $T_1(\lambda_{2 \rightarrow 1} \bar{T}_2 - \bar{Q}_2) - (Q_1 + T_1 \lambda_{2 \rightarrow 1})$
 $\bar{Q}_2 T_1 + Q_1$
[U+0001 control character]<
<
m
-
R
 $Q_1(\bar{Q}_2 - \lambda_{2 \rightarrow 1})$
 $Q_1(\lambda_{2 \rightarrow 1} \bar{T}_2 - \bar{Q}_2) + \lambda_{2 \rightarrow 1}(Q_1 + T_1 \lambda_{2 \rightarrow 1})$
1
 ∂
Then ∂u
 $R(w, \sigma) > 0$, for all u , σ_2 . Furthermore, the constraint set is feasible regardless of the primitives.

We can now do the same thing for the first state, assuming that $w_1(\tau)$ is of the form $w_1(\tau) = m_1\tau + zq_{1 \rightarrow 2}(\tau)$, where now $z \leq 0$ and $m = R_2$. In the next lemma, we consider u an upper endpoint of σ_1 , and so ∂u
 $R(w = \{w_1, w_2\}, \sigma = \{\sigma_1, \sigma_2\})$ is a derivative with respect to an upper endpoint of σ_1 . Then,

[Next verbatim source excerpt]

Theorem 3. Let $R_1 < R_2$ be target earning rates during non-surged and surge states, respectively. There exist prices $w = \{w_1, w_2\}$ of the form $w_i(\tau) = m_i\tau + z_i q_{i \rightarrow j}(\tau)$, where $m_1, m_2, z_2 \geq 0$ (but z_1 may be either positive or negative), such that the optimal driver policy is to accept every trip in the surge state and all trips up to a certain length in the non-surge state.

[Next verbatim source excerpt]

Proof. Proof.
Suppose we have $w_2(u) = mu + zq_{2 \rightarrow 1}(u)$, for some $m > R_1$, $z \geq 0$.

Expanded PaperInterface specification

```
V (arrivalRate lowerEndpoint u T1 Q1 T2 Q2 Tbar2 Qbar2 switch21 switch12 m R1 z ratio : ℝ) (density : ℝ → ℝ)
(harrival_pos : 0 < arrivalRate) (hdensity_pos : 0 < density u) (hq1_pos : 0 < Q1)
(hden :
  GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch21 lowerEndpoint density
    (GN21DriverSurgePricing.gn21SwitchProb switch21 switch12) u *
    T1 +
    Q1 * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u ≠
    0)
(hq_int :
  IntervalIntegrable (fun τ => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ)
    MeasureTheory.volume lowerEndpoint u)
(hq_meas :
  StronglyMeasurableAtFilter (fun τ => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ) (nhds u)
    MeasureTheory.volume)
(hq_cont : ContinuousAt (fun τ => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ) u)
(hw_int :
  IntervalIntegrable (fun τ => GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ)
    MeasureTheory.volume lowerEndpoint u)
(hw_meas :
  StronglyMeasurableAtFilter
    (fun τ => GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ) (nhds u)
    MeasureTheory.volume)
(hw_cont :
  ContinuousAt (fun τ => GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ) u)
(ht_int : IntervalIntegrable (fun τ => τ * density τ) MeasureTheory.volume lowerEndpoint u)
(ht_meas : StronglyMeasurableAtFilter (fun τ => τ * density τ) (nhds u) MeasureTheory.volume)
(ht_cont : ContinuousAt (fun τ => τ * density τ) u),
GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch21 lowerEndpoint density
  (GN21DriverSurgePricing.gn21SwitchProb switch21 switch12) u =
  Q2 →
  GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u = T2 →
  GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density
    (GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12) u =
    m * (T2 - 1) + z * (Q2 - switch21) →
  GN21DriverSurgePricing.lemma9StructuredBounds ratio T1 Q1 Tbar2 Qbar2 switch21 →
  0 ≤ z →
  z = ratio * (m - R1) →
  0 < m - R1 →
  0 ≤ R1 →
  0 ≤ T1 →
  0 < switch21 →
  0 < switch21 + switch12 →
  0 < u →
  0 ≤ switch21 * T2 - Q2 →
  switch21 * T2 - Q2 ≤ switch21 * Tbar2 - Qbar2 →
  switch21 < Q2 →
```

```

Q2 ≤ Qbar2 →
0 <
  (GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths
   arrivalRate switch21 lowerEndpoint u Q1 T1 (R1 * T1) density
   (GN21DriverSurgePricing.gn21SwitchProb switch21 switch12)
   (GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12)
   harriaval_pos hdensity_pos hQ1_pos hden hq_int hq_meas hq_cont hw_int hw_meas
   hw_cont ht_int ht_meas ht_cont).derivativeValue

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma9_surge_derivative_positive_of_acceptAll_bounds

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 9 fixes the non-surge policy, applies the displayed surge-state price and accept-all bounds, and concludes positivity of the relevant surge derivative. The transparent Spec records those fixed quantities, bounds, and derivative conclusion without suppressing the interval conditions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims (continued)

Review row 11

Source locator: cited publication, lines 498-548

Verbatim source input

Theorem 1 establishes that, in a single-state model with Poisson job arrivals, the length of the job is not important, only the hourly rate while busy on the job. The optimal cw in the policy is

This practice is not consistent across marketplaces and locations. For example, in California as of January 2020, Uber shows the driver the destination and payment estimate at request time. Incentive compatible pricing is an important stepping stone to showing this information.

We note that destination discrimination is against Uber's guidelines and could lead to deactivation (Uber, 2019a).

Lyft has recently experimented with paying drivers for the time it takes to pick up the rider (Auerbach, 2019).

[form-feed in source extraction]

)

not necessarily $cw = \sup w(\tau)$

τ : drivers must trade off the earnings rate while on a trip with their utilization rate; the more trips that a driver rejects, the longer the wait for an acceptable trip. In the appendix we prove the result by, starting at an arbitrary policy σ , making changes to the policy that increase the earnings rate while on a job without decreasing the utilization rate. Thus, each such change improves the reward $R(w, \sigma)$, and the sequence of changes results in a policy of the above form, for some threshold c_0 . Then, this threshold c_0 can be optimized, leading to an optimal policy of this form.

An immediate corollary of Theorem 1 is that $w(\tau) = m\tau$, for $m > 0$, is IC. In other words, if the

)

platform pays a constant rate $w(\tau)$

$\tau = m$ to busy drivers, then in the single-state model it is in the driver's best interest to accept every trip. This result is driven by the following insight for Poisson arrivals: while receiving long trip requests is more beneficial to drivers in the single-state setting as they increase one's utilization rate (the driver is busy for a longer time until the next open period), rejecting short trips to cherry-pick long trips decreases utilization by the same amount.¹⁰ Further note that, given an earnings rate target R , calculating the multiplier m and thus an IC pricing policy is trivial.

On the other hand, affine pricing may not be incentive compatible because short trips are worth

)

a

more per unit time than are long trips: $w(\tau)$

$\tau = m + \tau$. The optimal policy may thus be to accept

*

trips in $\sigma = (0, T)$ for some T . However, our next proposition establishes that affine pricing is incentive compatible if the additive component stays small enough as a function of the request arrival rate:

With a single state, $w(\tau) = m\tau + a$ is incentive compatible if $0 \leq a \leq m$

λ .

The sufficient condition has a simple intuition: when open, the expected amount of time the driver must wait for the next request is λ^{-1} ; if on-trip time is valued at m per unit-time, then with

$a = m$

λ the additive component can be interpreted as paying for the driver's expected waiting time. Thus, while a driver may earn more per hour for a short trip than a long trip with affine pricing, such a short trip is not worth the time the driver must wait for the next trip request. We further note that the condition in the proposition is not a necessary one; however, deriving necessary and sufficient conditions in closed form requires specifying the trip distribution F .

Expanded PaperInterface specification

```
∀ (mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate m a : ℝ),
  GN21DriverSurgePricing.singleStateTripMass mu GN21DriverSurgePricing.acceptAllPolicy = 1 →
  MeasureTheory.IntegrableOn (fun τ => τ) GN21DriverSurgePricing.acceptAllPolicy mu →
  0 < arrivalRate →
  0 ≤ a →
  a ≤ m / arrivalRate →
  GN21DriverSurgePricing.singleStateMeasurableIncentiveCompatible
    (GN21DriverSurgePricing.affineSingleStateRenewalReward mu arrivalRate m a)
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_proposition3_1_affine_single_state_ic

Recorded source-to-Spec screening Verdict: matches

Reason: Proposition 3.1 states that affine payment proportional to trip duration is incentive compatible. The Spec uses the paper's affine one-state reward and concludes the accept-all incentive-compatibility relation under the displayed positive-arrival and parameter domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 455-520

Verbatim source input

freely reject it without penalty. Drivers often cannot see the rider's destination or the trip length until they pick up the rider (but they can reject a request based on the pick-up time to the rider, without penalty).⁷ Some drivers call ahead to find out the rider's destination or even cancel the trip at the pick-up location, creating negative experiences for both the rider and the driver.⁸ Our notion of incentive compatibility is ex-post, in which drivers would accept all trips even knowing the trip length. This notion is stronger than an ex-ante setting in which the trip length is not revealed to drivers. Furthermore, in practice, jobs have two components: the time it takes to pick up the rider, and the time while the rider is in the driver's vehicle – and the former component is typically unpaid.⁹ Our model combines these two components into an overall trip length, which determines payments.

Markovian surge and model limitations. In practice, surge has strong intra-day patterns – for example, rush hours have higher average surge values, cf. Appendix Figure 9b. However, evolution of surge on finer time scales, on the level of drivers' individual trip decisions, is more volatile and believably Markovian, cf. Appendix Figure 9c. Our theoretical model assumes that surge is Markovian and binary and the response of a single driver, and further ignores spatial effects. We discuss such issues in Sections A.3 and B.1, and our empirical analysis in Section 6 provides evidence that our insights extend to practice despite these theoretical limitations.

3

Incentive compatibility with affine pricing

In this section, we study the incentive compatibility of affine pricing. In Section 3.1, we first characterize the driver's best-response strategy with respect to any pricing function w in the single-state model. We then observe $\hookrightarrow$ that multiplicative pricing, a special case of affine pricing where $w(\tau) = m\tau$, is incentive compatible. In contrast, in Section 3.2, we show that in the dynamic model, multiplicative pricing may no longer be incentive compatible. We further derive the structure of optimal driver policies in each state with respect to affine or multiplicative pricing, which will enable numerical study of the incentive compatibility properties of additive and multiplicative surge in Section 5. Section 3.3 discusses the key differences in the two models, setting up Section 4 where we derive incentive compatible pricing functions for the dynamic model.

3.1

Single-state model: multiplicative pricing is incentive compatible

Our first result is a simple optimal driver policy in the single-state model.

Theorem

1. With σ

a single state, for each w there exists a constant $c_w \in \mathbb{R}_+$ such that the policy

n

$w(\tau)$

*

$\sigma = \tau : \tau \geq c_w$ is optimal for the driver with respect to w .

Theorem 1 establishes that, in a single-state model with Poisson job arrivals, the length of the job is not important, only the hourly rate while busy on the job. The optimal c_w in the policy is

7

This practice is not consistent across marketplaces and locations. For example, in California as of January 2020, Uber shows the driver the destination and payment estimate at request time. Incentive compatible pricing is an important stepping stone to showing this information.

8

We note that destination discrimination is against Uber's guidelines and could lead to deactivation (Uber, 2019a).

9

Lyft has recently experimented with paying drivers for the time it takes to pick up the rider (Auerbach, 2019).

9

[form-feed in source extraction]

)

not necessarily $c_w = \sup w(\tau)$

τ : drivers must trade off the earnings rate while on a trip with their

utilization rate; the more trips that a driver rejects, the longer the wait for an acceptable trip. In the appendix we prove the result by, starting at an arbitrary policy σ , making changes to the policy that increase the earnings rate while on a job without decreasing the utilization rate. Thus, each such change improves the reward $R(w, \sigma)$, and the sequence of changes results in a policy of the above form, for some threshold c_0 . Then, this threshold c_0 can be optimized, leading to an optimal policy of this form.

An immediate corollary of Theorem 1 is that $w(\tau) = m\tau$, for $m > 0$, is IC. In other words, if the

Expanded PaperInterface specification

```
∀ (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ)
(w : GN21DriverSurgePricing.PricingFunction),
(Measurable fun τ => w τ / τ) →
  (∀ (τ : GN21DriverSurgePricing.TripLength), 0 < τ → 0 ≤ w τ / τ) →
    μ GN21DriverSurgePricing.acceptAllPolicy ≠ τ →
      MeasureTheory.IntegrableOn w GN21DriverSurgePricing.acceptAllPolicy μ →
        MeasureTheory.IntegrableOn (fun τ => τ) GN21DriverSurgePricing.acceptAllPolicy μ →
          0 < arrivalRate →
            ∃ c,
              0 ≤ c ∧
                ∃ σ,
                  GN21DriverSurgePricing.thresholdRatePolicy w c σ ∧
                    GN21DriverSurgePricing.singleStateMeasurableOptimal
                      (GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w) σ
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem1_single_state_threshold_best_response

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 1 states that a single-state optimum can be selected as a threshold policy. The expanded Spec gives an explicit cutoff and policy satisfying the source threshold relation and concludes optimality for the one-state renewal reward under the stated measurable and integrability domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 560-656

Verbatim source input

Theorem 2. If $w = \{w_1, w_2\}$, there exists an optimal policy $\sigma = \{\sigma_1, \sigma_2\}$ (i.e., that maximizes $R(w, \sigma)$), defined with parameters $t_1, t_2, t_3, t_4, t_5, t_6 \in [0, \infty) \cup \{\infty\}$, such that

- Non-surge state driver optimal policy σ_1 :
 - If w_1 is multiplicative or positive affine, σ_1 rejects long trips, i.e., $\sigma_1 = (0, t_1)$.
 - If w_1 is negative affine, σ_1 rejects short and long trips, i.e., $\sigma_1 = (t_2, t_3)$.

10

This insight is similar to a result of Kamble (2019); however, in our setting the driver's strategy σ is a subset of R^+ denoting the job requests accepted, as opposed to a discrete set of prices charged. Further, in our settings the driver responds to the platform's prices instead of setting prices, enabling a wider range of IC pricing mechanisms.

10

[form-feed in source extraction]

- Surge state driver optimal policy σ_2 :
 - If w_2 is multiplicative or negative affine, σ_2 rejects short trips, i.e., $\sigma_2 = (t_4, \infty)$.
 - If w_2 is positive affine, σ_2 rejects medium length trips, i.e., $\sigma_2 = (0, t_5) \cup (t_6, \infty)$.

Furthermore, there exist settings where t_i 's take positive finite values, and in which multiplicative pricing is not incentive compatible in either state. Finally, only policies of the appropriate form as indicated (up to differences of measure 0) can be optimal. We discuss the intuition in the next section. In the appendix, we prove the result for each case as follows: fixing σ_j for $j \neq i$, we start with an arbitrary open set $\sigma_i = (u, \infty)$ or $(-\infty, u)$, recalling that open sets can be written as a countable union of such disjoint intervals. Then, we find $\partial u \partial_k R(w, \sigma)$, the derivative of the set function $R(w, \sigma)$ with respect to one of the interval upper end-points of σ_i , i.e., u_k . This derivative is the infinitesimal change in the overall reward if σ_i is expanded by increasing u_k , and it has useful properties. In the surge state with multiplicative pricing, for example, $\partial u \partial_k R(w, \sigma)$ has the same sign as a function that is increasing in u , for each fixed σ . With affine pricing, it has the same sign as a quasi-convex (positive affine in the surge state) or quasiconcave (negative affine in the non-surge state) function in u , for a fixed σ . Such properties enable constructing a sequence of changes to σ_i that each do not decrease the reward $R(w, \sigma)$, with the limit being a policy of the appropriate form. In particular, we can show that any policy that is not of the appropriate form above has $\partial u \partial_k R(w, \sigma) > 0$ for some u_k , allowing local improvements until adjacent intervals (u_k, u_{k+1}) can be combined or expanded to infinity. The numerics in Section 5 provide examples in which multiplicative pricing is not incentive compatible, i.e., where policies of the form above with positive finite constants strictly increase driver earnings over the driver policy that accepts all trip requests. The results of rejecting long trips in non-surge (and short trips in surge) extend to arbitrary functions where $w_1(\tau)$ is non-increasing (respectively, $w_2(\tau)$ is non-decreasing). The other two results do not hold with such generality, as the behavior of the derivative may be arbitrarily complex.

3.3

Why is multiplicative surge pricing not incentive compatible?

"I thoroughly dislike short trips ESPECIALLY when I'm picking up in a waning surge zone"
Anonymous driver

What explains the difference between multiplicative pricing being incentive compatible in the singlestate model but not in the

dynamic model? In the latter, a driver's policy affects not just their earnings while they are busy, but also the fraction of time during which they are busy during the lucrative surge state. In particular, it turns out, accepting short trips during surge may reduce the amount of time that a driver is on a surge trip! Appendix Figure 7 shows in an example how the fraction of time in the surge state $\mu_2(\sigma)$ changes as a function of how many short trips the driver rejects.

The anonymous driver we quote above identifies the key effect: when surge is short-lived, a driver may only have the chance to complete one surge trip before it ends. Thus, the driver may be better off waiting to receive a longer trip request, as with multiplicative surge they are paid a higher rate for the full duration of the longer trip. (Of course, there is a trade-off as rejecting too many trip requests risks not receiving any acceptable request before surge ends). In the surge state, then, multiplicative pricing does not compensate drivers enough to accept short trips that may reduce their future surge earnings. In the non-surge state, analogously, multiplicative pricing under-values long trips that may prevent taking advantage of a future surge.

11

[form-feed in source extraction]

Affine pricing is a first, reasonable attempt at fixing these issues. In the surge state, the additive value makes the previously under-valued short trips comparatively more valuable, as the earnings per unit time $w_2(\tau) = m_2 + a_2\tau$ (with $a_2 > 0$) are now higher for short trips. Unfortunately, with such pricing the structure for the surge optimal policy becomes $\sigma_2 = (0, t_5) \cup (t_6, \infty)$ – if the values m_2, a_2 are not balanced correctly, the additive value is enough to make accepting extremely short trips $(0, t_5)$ profitable; for medium-length trips $\tau \in (t_5, t_6)$, however, the additive value is not large enough to make up for the fact that accepting the trip prevents accepting another surged trip before surge ends. Similarly, negative affine pricing in the non-surge state, $w_1(\tau) = m_1\tau + a_1$, (with $a_1 < 0$) is now too harsh on very short trips but potentially not enticing enough for long trips. Next, we fix these issues and construct incentive compatible pricing schemes for our dynamic model. Then, in Section 5 we leverage structural results derived here to numerically compare the incentive compatibility of additive and multiplicative surge.

4

Incentive Compatible Surge Pricing

We now present our main result, regarding the structure of incentive compatible pricing in the dynamic model. To this aim, in Section 4.1, we characterize $\mu_i(\sigma)$, how much time the driver

spends in each state. In Section 4.2, we present incentive compatible prices, under a condition R1 on the ratio of per-state earning rate constraints, R. Section 4.3 discusses an intuition of the IC pricing structure in terms of the driver's opportunity cost.

4.1

Transition probabilities and expected time spent in each state

The expected fraction of time spent in each state, $\mu_i(\sigma)$, depends both on the evolution of the world state and the trips a driver accepts. To quantify the effects previewed in Section 3.3, we first analyze the evolution of the world state CTMC.

Expanded PaperInterface specification

```
((0 < 2 ^
  GN21DriverSurgePricing.rejectsLongTrips 2
  (GN21DriverSurgePricing.theorem2BothStatesContinuousNonsurgeDeviation 0)) ^
GN21DriverSurgePricing.dynamicProfitableDeviation
  (GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional
   GN21DriverSurgePricing.theorem2BothStatesContinuousMu
   GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun i =>
   GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i))
  GN21DriverSurgePricing.theorem2BothStatesContinuousNonsurgeDeviation) ^
(0 < 2 ^
  GN21DriverSurgePricing.rejectsShortTrips 2
  (GN21DriverSurgePricing.theorem2BothStatesContinuousSurgeDeviation 1)) ^
GN21DriverSurgePricing.dynamicProfitableDeviation
  (GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional
   GN21DriverSurgePricing.theorem2BothStatesContinuousMu
   GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun i =>
   GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i))
  GN21DriverSurgePricing.theorem2BothStatesContinuousSurgeDeviation) ^
~GN21DriverSurgePricing.dynamicIncentiveCompatible
  (GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional GN21DriverSurgePricing.theorem2BothStatesContinuousMu
   GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun i =>
   GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i))
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem2_multiplicative_positive_finite_cutoff_not_ic_both_states

Recorded source-to-Spec screening Verdict: matches

Reason: The selected Theorem 2 source instance supplies a multiplicative pricing rule with profitable positive finite-cutoff deviations in both states and therefore fails dynamic incentive compatibility. The Spec states that concrete two-state instance and both deviations directly, rather than using the theorem name as evidence.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix source claims (continued 2)

Review row 14

Source locator: cited publication, lines 3859-3880

Verbatim source input

Theorem 4. Consider pricing function $w = \{w_1, w_2\}$, where $i = 2$ is the surge state as defined. Then, there exists an optimal policy $\sigma = \{\sigma_1, \sigma_2\}$ that maximizes $R(w, \sigma)$, with the following properties.

- Non-surge state driver optimal policy σ_1 :
 - If $w_1(\tau) = m_1 \tau + a_1$, for $a_1 \geq 0$, then $\sigma_1 = (\theta, t_1)$, for some $t_1 \in [0, \infty) \cup \{\infty\}$.
 - If $w_1(\tau) = m_1 \tau - a_1$, for $a_1 > 0$, then $\sigma_1 = (t_2, t_3)$, for some $t_2, t_3 \in [0, \infty) \cup \{\infty\}$.

∂
– If w_1 such that ∂u
 $R(w, \sigma \theta = \{\sigma_{10}, \sigma_{20}\}) > 0$ for all $\sigma \theta$, where u is an upper endpoint of an interval that makes up σ_{10} , then $\sigma_1 = (\theta, \infty)$.

- Surge state driver optimal policy σ_2 :
 - If $w_2(\tau) = m_2 \tau - a_2$, for $a_2 \geq 0$, then $\sigma_1 = (t_4, \infty)$, for some $t_4 \in [0, \infty)$.
 - If $w_2(\tau) = m_2 \tau + a_2$, for $a_2 > 0$, then $\sigma_1 = (\theta, t_5) \cup (t_6, \infty)$, for some $t_5, t_6 \in [0, \infty) \cup \{\infty\}$.

∂
– If w_2 such that ∂u
 $R(w, \sigma \theta = \{\sigma_{10}, \sigma_{20}\}) > 0$ for all $\sigma \theta$, where u is an upper endpoint of an interval that makes up σ_{20} , then $\sigma_2 = (\theta, \infty)$.

Furthermore, only policies of the given forms can be optimal.

[Next verbatim source excerpt]

[form-feed in source extraction]

- Start with some arbitrary policy $\sigma = \{\sigma_1, \sigma_2\}$.
- With assumption on the surge state providing higher potential earnings, replace σ_2 with a policy that provides higher earnings in state 2 than σ_1 does in state 1, without decreasing total reward.
- Using Lemma 5, replace σ_1 with policy of the appropriate form, without decreasing total reward.
- Using Lemma 5, replace σ_2 with policy of the appropriate form, without decreasing total reward.

∂
Let $r(u, i, w, \sigma)$ be as defined in Lemma 6, a function that has the same sign as ∂u
 $R(w, \sigma)$,
where u is an upper endpoint of an interval that is part of σ_i . Recall that, above, we show

- (Remark 1). $\Delta(\sigma_i, \sigma_j) > 0$ and $w_i(\tau)$ non-decreasing implies $r(u, i, w, \sigma)$ strictly increasing in $u \in \sigma_i$.
 - (Remark 1). $\Delta(\sigma_i, \sigma_j) < 0$ and $w_i(\tau)$ non-increasing implies $r(u, i, w, \sigma)$ strictly decreasing in $u \in \sigma_i$.
 - (Lemma 7). $w(\tau) = m\tau + a$ for $m, a > 0$ and $\Delta(\sigma_i, \sigma_{-i}) \geq 0$ implies $r(u, i, w, \sigma)$ is strictly quasi-convex in $u \in \sigma_i$.
 - (Lemma 8). $w(\tau) = m\tau - a$ for $m, a > 0$ and $\Delta(\sigma_i, \sigma_{-i}) \leq 0$ implies $r(u, i, w, \sigma)$ is strictly quasi-concave in $u \in \sigma_i$.
- We need to show that there exists a σ of the appropriate form such that $R(w, \sigma) \geq R(w, \sigma \theta)$, for all $\sigma \theta$.

Start with arbitrary $\sigma \theta = \{\sigma_{10}, \sigma_{20}\}$ where $\sigma_{10}, \sigma_{20} \subseteq \mathbb{R}^+$ are open, measurable sets, but not of the correct form in the theorem statement. Invoking Lemma 5 as appropriate, we construct a sequence of changes to $\sigma \theta$ such that the overall reward does not decrease with each change, and the sequence ends with a policy consistent with the theorem statement.

Step A First, if $R_2(\sigma_{20}) < R_1(\sigma_{10})$, then we replace σ_{20} with a policy σ_{2A} such that $R_2(\sigma_{2A}) > R_1(\sigma_1)$, $\forall \sigma_1$.

Let σ_{2A} be such that $\Delta(\sigma_{2A}, \sigma_{100}) > 0$, for all σ_{100} open and measurable. Such σ_{2A} exists by the definition of the surge state (we in fact define the surge state so that such σ_{2A} exists). Then, let $\sigma_A, \{\sigma_{1A} = \sigma_{10}, \sigma_{2A}\}$.

Note that $R(w, \sigma_A) \geq R(w, \sigma \theta)$: time spent earning reward at the rate of $R_2(w_2, \sigma_{20})$ is replaced by time spent earning at rate $R_1(w_1, \sigma_{10})$ or earning at rate $R_2(w_2, \sigma_{2A})$; time spent earning at $R_1(w_1, \sigma_{10})$ may be replaced by time earning at rate $R_2(w_2, \sigma_{2A})$.

Step B Now, we replace σ_{1A} with a policy that is of the appropriate form.

Let $\hat{R}(\sigma_1), R(w, \{\sigma_1, \sigma_{2A}\})$. By Lemma 5, there exists σ_{1B} such that $R(w, \{\sigma_{1B}, \sigma_{2A}\}) \geq R(w, \{\sigma_{1A}, \sigma_{2A}\})$, and σ_{1B} is of the required form. The inequality is strict if σ_{1A} is not of the required form.

Note that all the assumptions of Lemma 5 are met for each appropriate case: σ_{2B} such that $\Delta(\sigma_{2B}, \sigma_{10}) > 0, \forall \sigma_{10}$, and so $r(u, i, w, \sigma)$ remains decreasing or strictly quasi-concave as necessary.

Let $\sigma_B, \{\sigma_{1B}, \sigma_{2B} = \sigma_{2A}\}$.

59

[form-feed in source extraction]

Step C Now, we replace σ_{2B} with a policy that is of the appropriate form.

Let $\hat{R}(\sigma_2), R(w, \{\sigma_{1B}, \sigma_2\})$.

By Lemma 5, there exists σ_{2C} such that $R(w, \{\sigma_{1B}, \sigma_{2C}\}) \geq R(w, \{\sigma_{1B}, \sigma_{2B}\})$, and σ_{2C} is of the required form according to the table. The inequality is strict if σ_{2B} is not of the required form. As before, all the assumptions of Lemma 5 are met for each appropriate case. σ_{2B} such that $\Delta(\sigma_{2B}, \sigma_{10}) > 0, \forall \sigma_{10}$, and so $r(u, i, w, \sigma)$ remains strictly increasing / strictly quasi-convex in u for a fixed σ .

Let $\sigma_C, \{\sigma_{1C} = \sigma_{1B}, \sigma_{2C}\}$.

Thus, we have constructed $\sigma^* = \{\sigma_{1^*} = \sigma_{1C}, \sigma_{2^*} = \sigma_{2C}\}$ such that $\sigma_{1^*}, \sigma_{2^*}$ correspond to theorem

statement for the appropriate cases, respectively, and $R(w, \sigma *) \geq R(w, \sigma)$, for all $\sigma = \{\sigma_1, \sigma_2\}$ where $\sigma_1, \sigma_2 \subseteq R^+$ are open, measurable sets, with the inequality strict if σ is not of the required form.

Expanded PaperInterface specification

```

∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) [MeasureTheory.NoAtoms (mu 0)]
[MeasureTheory.NoAtoms (mu 1)] [MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)]
[(mu 0).InnerRegularCompactLTTTop] [(mu 1).InnerRegularCompactLTTTop] (arrival : Fin 2 → ℝ) (switch12 switch21 : ℝ)
(w : Fin 2 → GN21DriverSurgePricing.PricingFunction) (shape : Fin 2 → GN21DriverSurgePricing.Lemma5DerivativeShape),
0 < arrival 0 →
0 < arrival 1 →
0 < switch12 →
0 < switch21 →
  Measurable (w 0) →
  Measurable (w 1) →
    MeasureTheory.IntegrableOn (fun tau => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
    MeasureTheory.IntegrableOn (fun tau => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
    MeasureTheory.IntegrableOn (w 0) GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
    MeasureTheory.IntegrableOn (w 1) GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
    GN21DriverSurgePricing.gn21SourceSurgeStateDominance mu arrival w →
    GN21DriverSurgePricing.gn21Theorem4NonsurgeLiteralEndpointPriceCase
      (GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21 w)
      w shape →
    GN21DriverSurgePricing.gn21Theorem4SurgeLiteralEndpointPriceCase
      (GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21
      w)
      w shape →
    (∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
      GN21DriverSurgePricing.dynamicFeasibleOpenPolicy rho →
        ∀ (i : Fin 2) (tau : GN21DriverSurgePricing.TripPolicy),
          GN21DriverSurgePricing.GN21SymmDiffContinuousAt (mu i)
            (fun policy =>
              GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12
              switch21 w (Function.update rho i policy))
            tau) →
        (∃ rho,
          GN21DriverSurgePricing.dynamicOpenOptimal
            (GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12
            switch21 w)
            rho ∧
          GN21DriverSurgePricing.lemma5SourcePolicyForm (shape 0) (rho 0) ∧
          GN21DriverSurgePricing.lemma5SourcePolicyForm (shape 1) (rho 1)) ∧
        ∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
          GN21DriverSurgePricing.dynamicOpenOptimal
            (GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12
            switch21 w)
            rho →
          GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere (mu 0) (shape 0)
            (rho 0) ∧
          GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere (mu 1) (shape 1)
            (rho 1))
  )

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem4_full_structural_policy_forms_direct

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 4 asserts existence of an open measurable optimum and the listed structural price-policy forms for every such optimum. The Spec retains the five response-shape cases, the source surge-state model, open measurable domain, and the existence-and-universal structural conclusion; the zero-density convention is represented by the source's stated almost-everywhere policy semantics, not by a full-support assumption.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims (continued 2)

Review row 15

Source locator: cited publication, lines 264-285

Verbatim source input

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}_+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}_+$. When we write equalities with policies σ , we mean equality up to changes of measure $\emptyset$.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is

$$R(w, \sigma) = \liminf_{t \rightarrow \infty}$$

$$R(w, \sigma, t)$$

t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (\emptyset, \infty)$ in the single-state model or $\sigma = \{(\emptyset, \infty), (\emptyset, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

where $w_i(\tau) = m_i \tau + a_i$, with $m_i \geq 0$ (in the single-state model: $w(\tau) = m\tau + a$, with $m \geq 0$; we refer to the case with $a_i > 0$ ($a_i < 0$) as positive (negative) affine pricing). Such pricing functions

[Next verbatim source excerpt]

Theorem 2. If $w = \{w_1, w_2\}$, there exists an optimal policy $\sigma = \{\sigma_1, \sigma_2\}$ (i.e., that maximizes $R(w, \sigma)$), defined with parameters $t_1, t_2, t_3, t_4, t_5, t_6 \in [0, \infty) \cup \{\infty\}$, such that

- Non-surge state driver optimal policy σ_1 :
 - If w_1 is multiplicative or positive affine, σ_1 rejects long trips, i.e., $\sigma_1 = (\emptyset, t_1)$.
 - If w_1 is negative affine, σ_1 rejects short and long trips, i.e., $\sigma_1 = (t_2, t_3)$.

This insight is similar to a result of Kamble (2019); however, in our setting the driver's strategy σ is a subset of $\mathbb{R}_+$ denoting the job requests accepted, as opposed to a discrete set of prices charged. Further, in our settings the driver responds to the platform's prices instead of setting prices, enabling a wider range of IC pricing mechanisms.

10

[form-feed in source extraction]

- Surge state driver optimal policy σ_2 :
 - If w_2 is multiplicative or negative affine, σ_2 rejects short trips, i.e., $\sigma_2 = (t_4, \infty)$.

– If w_2 is positive affine, σ_2 rejects medium length trips, i.e., $\sigma_2 = (\emptyset, t_5) \cup (t_6, \infty)$. Furthermore, there exist settings where t_i 's take positive finite values, and in which multiplicative pricing is not incentive compatible in either state. Finally, only policies of the appropriate form as indicated (up to differences of measure $\emptyset$) can be optimal.

We discuss the intuition in the next section. In the appendix, we prove the result for each case as follows: fixing σ_j for $j \neq i$, we start with an arbitrary open set $\sigma_i = \cup_{k \in \mathbb{N}}$

$k(\tau_k, \tau_{k+1})$, recalling that

open sets can be written as a countable union of such disjoint intervals. Then, we find $\partial_u R(w, \sigma)$, the derivative of the set function $R(w, \sigma)$ with respect to one of the interval upper end-points of σ_i , i.e., τ_k . This derivative is the infinitesimal change in the overall reward if σ_i is expanded by increasing τ_k , and it has useful properties. In the surge state with multiplicative pricing, for

example, ∂_u

$R(w, \sigma)$ has the same sign as a function that is increasing in u , for each fixed σ . With affine pricing, it has the same sign as a quasi-convex (positive affine in the surge state) or quasiconcave (negative affine in

the non-surge state) function in u , for a fixed σ . Such properties enable constructing a sequence of changes to σ_i that each do not decrease the reward $R(w, \sigma)$, with the limit being a policy of the appropriate form. In particular, we can show that any policy that is not of the appropriate form above has $\partial_u R(w, \sigma) > 0$ for some τ_k , allowing local improvements until adjacent intervals (τ_k, τ_{k+1}) , (τ_{k+1}, τ_{k+2}) can be combined or expanded to infinity. The numerics in Section 5 provide examples in which multiplicative pricing is not incentive compatible, i.e., where policies of the form above with positive finite constants strictly increase driver earnings over the driver policy that accepts all trip requests.

The results of rejecting long trips in non-surge (and short trips in surge) extend to arbitrary functions where $w_1(\tau)$ is non-increasing (respectively, $w_2(\tau)$ is non-decreasing). The other two results do not hold with such generality, as the behavior of the derivative may be arbitrarily complex.

3.3

Why is multiplicative surge pricing not incentive compatible?

"I thoroughly dislike short trips ESPECIALLY when I'm picking up in a waning surge zone"

Anonymous driver

What explains the difference between multiplicative pricing being incentive compatible in the singlestate model but not in the

dynamic model? In the latter, a driver's policy affects not just their earnings while they are busy, but also the fraction of time during which they are busy during the lucrative surge state. In particular, it turns out, accepting short trips during surge may reduce the amount of time that a driver is on a surge trip! Appendix Figure 7 shows in an example how the

fraction of time in the surge state $\mu_2(\sigma)$ changes as a function of how many short trips the driver rejects.

The anonymous driver we quote above identifies the key effect: when surge is short-lived, a driver may only have the chance to complete one surge trip before it ends. Thus, the driver may be better off waiting to receive a longer trip request, as with multiplicative surge they are paid a higher rate for the full duration of the longer trip. (Of course, there is a trade-off as rejecting too many trip requests risks not receiving any acceptable request before surge ends). In the surge state, then, multiplicative pricing does not compensate drivers enough to accept short trips that may reduce their future surge earnings. In the non-surge state, analogously, multiplicative pricing under-values long trips that may prevent taking advantage of a future surge.

11

[form-feed in source extraction]

Affine pricing is a first, reasonable attempt at fixing these issues. In the surge state, the additive value makes the previously under-valued short trips comparatively more valuable, as the earnings per unit time $w_2(\tau) = m_2 + a_2\tau$ (with $a_2 > 0$) are now higher for short trips. Unfortunately, with such pricing the structure for the surge optimal policy becomes $\sigma_2 = (0, t_5) \cup (t_6, \infty)$ – if the values m_2, a_2 are not balanced correctly, the additive value is enough to make accepting extremely short trips $(0, t_5)$ profitable; for medium-length trips $\tau \in (t_5, t_6)$, however, the additive value is not large enough to make up for the fact that accepting the trip prevents accepting another surged trip before surge ends. Similarly, negative affine pricing in the non-surge state, $w_1(\tau) = m_1\tau + a_1$, (with $a_1 < 0$) is now too harsh on very short trips but potentially not enticing enough for long trips. Next, we fix these issues and construct incentive compatible pricing schemes for our dynamic model. Then, in Section 5 we leverage structural results derived here to numerically compare the incentive compatibility of additive and multiplicative surge.

4

Incentive Compatible Surge Pricing

We now present our main result, regarding the structure of incentive compatible pricing in the dynamic model. To this aim, in Section 4.1, we characterize $\mu_i(\sigma)$, how much time the driver spends in each state. In Section 4.2, we present incentive compatible prices, under a condition R1

on the ratio of per-state earning rate constraints, R

. Section 4.3 discusses an intuition of the IC

2

pricing structure in terms of the driver's opportunity cost.

4.1

Transition probabilities and expected time spent in each state

The expected fraction of time spent in each state, $\mu_i(\sigma)$, depends both on the evolution of the world state and the trips a driver accepts. To quantify the effects previewed in Section 3.3, we first analyze the evolution of the world state CTMC.

[Next verbatim source excerpt]

We assume throughout:

- Distribution of jobs F, F_i is a continuous probability measure, i.e., f, f_i bounded.
- There exists a policy in state 2 that dominates state 1: $\exists \sigma_2$ such that $\Delta(\sigma_2, \sigma_1) > 0, \forall \sigma_1 \in (0, \infty)$.
- σ, σ_i constrained to be measurable with respect to F, F_i , and σ_i are open.

Expanded PaperInterface specification

```

V (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) [MeasureTheory.NoAtoms (mu 0)]
  [MeasureTheory.NoAtoms (mu 1)] [MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)]
  (arrival : Fin 2 → ℝ) (switch12 switch21 : ℝ) (m : Fin 2 → ℝ),
  0 ≤ m 0 →
  0 ≤ m 1 →
  0 < arrival 0 →
  0 < arrival 1 →
  0 < switch12 →
  0 < switch21 →
  GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1 →
  GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1 →
  MeasureTheory.IntegrableOn (fun tau => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
  MeasureTheory.IntegrableOn (fun tau => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
  (GN21DriverSurgePricing.gn21SourceSurgeStateDominance mu arrival fun i =>
    GN21DriverSurgePricing.multiplicativePricing (m i)) →
  (∃ rho,
    GN21DriverSurgePricing.dynamicOpenOptimal
      (GN21DriverSurgePricing.gn21AggregateMultiplicativeDynamicReward mu arrival switch12
        switch21 m)
      rho ∧
    GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 0)
      (rho 0) ∧
    GN21DriverSurgePricing.rejectsShortTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 1)
      (rho 1) ∧
  V (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
  GN21DriverSurgePricing.dynamicOpenOptimal
    (GN21DriverSurgePricing.gn21AggregateMultiplicativeDynamicReward mu arrival switch12
      switch21 m)
    rho →
  GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 0)
    (rho 0) ∧
  GN21DriverSurgePricing.rejectsShortTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 1)
    (rho 1)

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem2_multiplicative_policy_shape_source_claim

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 2 gives the multiplicative-pricing policy shapes: an optimal open policy exists and every optimal open policy has the stated non-surge reject-long and surge reject-short cutoff forms up to null sets. The expanded Spec states both quantifier clauses and the continuous-law, positive-rate, and surge-state model domain used in the source.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix source claims (continued 3)

Review row 16

Source locator: cited publication, lines 3343-3386

Verbatim source input

Lemma 5. Consider a function $\hat{R}(\sigma)$ that maps open, measurable subsets $\sigma = \cup_{k=1}^{\infty} (k, u_k) \subseteq (0, \infty)$ to the non-negative reals, and probability measure F such that F is continuous, i.e. f is bounded.

Let $\partial_u \hat{R}(\sigma)$ denote the partial derivative of $\hat{R}$ with respect to an upper end-point u of the intervals that make up $\sigma = \cup_{k=1}^{\infty} (k, u_k)$, i.e., it is the infinitesimal gain in the value of $\hat{R}(\sigma)$ by adding u to the set σ . Suppose,

1. $\hat{R}(\sigma)$ is continuous in σ , and $\partial_u \hat{R}(\sigma)$ exists, for all σ and u .

2. $\partial_u \hat{R}(\sigma)$ is continuous in u , for each fixed σ .

3. $\partial_u \hat{R}(\sigma)$ is continuous in σ , for each fixed u .

Suppose that there exists a function $r(u, \sigma)$ that has the same sign as $\partial_u \hat{R}(\sigma)$, for all σ, u where

$f(u) > 0$. Consider any open measurable subset $\sigma \subseteq (0, \infty)$, where $F(\sigma) > 0$, and $\hat{R}(\sigma) > \hat{R}(\emptyset)$.

50

[form-feed in source extraction]

Then the following statement holds for each of the below specific cases: "Suppose $\exists [U+000F \text{ control character}] > 0$ s.t. $r(u, \sigma)$ is [Property] in u (for a fixed σ), for all σ such that $\hat{R}(\sigma) \geq \hat{R}(\sigma_0) - [U+000F \text{ control character}]$. Then, there exists a policy σ^* of the form [Form] such that $\hat{R}(\sigma^*) \geq \hat{R}(\sigma_0)$, and the inequality is strict unless σ_0 is also of the same form."

Property

$r(u, \sigma) > 0$ (is positive)
 $r(u, \sigma)$ is strictly increasing
 $r(u, \sigma)$ is strictly decreasing
 $r(u, \sigma)$ is strictly quasi-convex
 $r(u, \sigma)$ is strictly quasi-concave

Form

$\sigma^* = (0, \infty)$
 $\sigma^* = (k^*, \infty)$, for $k^* \in \mathbb{R}^+$
 $\sigma^* = (0, u^*)$, for $u^* \in \mathbb{R}^+ \cup \{\infty\}$
 $\sigma^* = (0, k^*) \cup (u^*, \infty)$, for $k^*, u^* \in \mathbb{R}^+ \cup \{\infty\}$
 $\sigma^* = (k^*, u^*)$, for $k^*, u^* \in \mathbb{R}^+ \cup \{\infty\}$

[Next verbatim source excerpt]

Proof. Proof. The facts that policies not of the appropriate form cannot be optimal follow directly from the respective structures of the derivatives, as we will show below. However, since the domain of the set function $\hat{R}$ is any subset of $\mathbb{R}^+$, we need to also prove existence of a maximizer of the appropriate form. The proof is structured around proving existence of a maximizer in each case, but we will point out where the given facts imply necessity of having the appropriate form.

The general approach is as follows: Start at subset $\sigma_0 \subseteq (0, \infty) = \cup_{k=1}^{\infty} (k, u_k)$, where the intervals are disjoint and $\zeta_k = (k, u_k)$ denotes the k th interval. (recall that any open subset of $\mathbb{R}$ can be uniquely written as the countable union of such disjoint intervals).

Then, do the following:

1. Create a sequence $\sigma_\delta \rightarrow \sigma_0$ (as $\delta \rightarrow 0$), where, for each δ , the set σ_δ is δ -close to σ_0 : $F((\sigma_0 \setminus \sigma_\delta) \cup (\sigma_\delta \setminus \sigma_0)) < \delta$.

2. Show that there exists a σ^* of the appropriate form (according to the property that holds above), such that $\hat{R}(\sigma_\delta) \leq \hat{R}(\sigma^*)$, $\forall \delta$.

By continuity of the set function $\hat{R}$, this implies that $\hat{R}(\sigma_0) \leq \hat{R}(\sigma^*)$.

The second step is the main proof step and the only one that depends substantially on the given property of the function $r(u, \sigma)$.

[U+0001 control character]

Step one: a sequence $\sigma_\delta \rightarrow \sigma_0$. Each σ_δ will be of the form $\sigma_\delta = (0, L) \cup \cup_{k=1}^K (k, u_k)$

$k=1 \dots K$ $(k, u_k) \cup (B, \infty)$,

for some K, B, L that depend on δ . We construct a σ_δ such that $F(\sigma_0 \setminus \sigma_\delta \cup \sigma_\delta \setminus \sigma_0) < \delta$ as follows:

• F is a finite (probability) measure, and so there exists K such that $F(\cup_{k=K+1}^{\infty} (k, u_k)) < \delta/2$.

(Since $F(\sigma) \leq 1$, it follows by the Cauchy condition).
• Let $B \in \mathbb{R}$ s.t. $F((B, \infty)) < \delta/4$. Let $L \in \mathbb{R}$ s.t. $F((0, L)) < \delta/4$. Such B, L exist by condition on F .

[U+0001 control character]

• Set $\sigma_\delta = (0, L) \cup \cup_{k=1}^K (k, u_k)$

$k=1 \dots K$ $(k, u_k) \cup (B, \infty)$.

• For convenience, we re-index the disjoint intervals $\{\zeta_k\}_{K+2}^{\infty}$ such that they are in increasing

order, i.e. $u_k > u_{k-1} \geq u_{k-2}, \forall k \geq 1$, starting at $(0, L)$, with the last interval (B, ∞) . If there exist any intervals such that $u_k = u_{k-1}$, replace them with the combined interval (u_{k-1}, u_k) . If $\{B, \infty\}$ overlaps with the last interval, combine them. Note that, by the suppositions, in each case $\exists \delta$ small enough such that $r(u, \sigma)$ maintains the appropriate property for all σ such that $\hat{R}(\sigma) \geq \hat{R}(\sigma_0)$, $\forall \delta < \delta_0$.

51

[form-feed in source extraction]

Step 2: showing that $\hat{R}(\sigma^*) \geq [U+0001 \text{ control character}] \hat{R}(\sigma_0)$, where σ^* is of the appropriate form. Now, starting at $\sigma = \sigma_0 = (0, L) \cup u_K$

$k=1 (u_k, u_{k+1}) \cup (B, \infty)$, we describe a sequence of modifications to σ , such that

each modification does not reduce the reward $\hat{R}(\sigma)$. The limit of this sequence of modifications is a policy σ^* of the appropriate form, regardless of the starting σ_0 .

We now carry out this step separately for each case. The general argument is that the properties force $r(u, \sigma)$ to be positive at certain points, which allows expanding the policy until a policy of the appropriate form is reached.

Let $r_L(u, \sigma) = -r(u, \sigma)$, i.e., it is a function that has the same sign as the derivative of a lower endpoint of σ (the same sign as the infinitesimal loss as removing the point u from the set σ).

$\frac{\partial}{\partial u} \hat{R}(\sigma)$ is positive in u , we can

Setting where $r(u, \sigma) > 0$ (is positive). By supposition that $\frac{\partial}{\partial u}$

increase u_1 (merging with other intervals) while increasing $\hat{R}(\sigma)$. Thus, we can keep increasing u_1 , and $u_1 \rightarrow B$, and so $\hat{R}((0, \infty)) \geq \hat{R}(\sigma_0)$. For any set $\sigma \neq (0, \infty)$, we can increase the reward by expanding an interval, and so it cannot be optimal. Thus, $(0, \infty)$ is the unique optimal set.

Setting where $r(u, \sigma)$ is strictly increasing.

Now, starting at $\sigma = \sigma_0$, the limit of the sequence of modifications is a policy $\sigma^* = (u^*, \infty)$.

By the supposition that $r(u, \sigma)$ strictly increasing in u , we have:

$r_L(u, \sigma)$ strictly decreasing
 $r_L(u_1, \sigma) \leq 0 \Rightarrow r(u_1, \sigma) > 0$

$\frac{\partial}{\partial u}$
 $\hat{R}(\sigma) \leq 0 \Rightarrow$

$\hat{R}(\sigma) > 0$

$\frac{\partial}{\partial u_1}$
 $\frac{\partial}{\partial u_1}$

$u_1 < u_1$

$r_L(u_1, \sigma) > 0 \Leftarrow r(u_1, \sigma) \leq 0$

$\frac{\partial}{\partial u}$
 $\hat{R}(\sigma) > 0 \Leftarrow$

$\hat{R}(\sigma) \leq 0$

$\frac{\partial}{\partial u_1}$
 $\frac{\partial}{\partial u_1}$

Case 1: $\exists \zeta_1, \zeta_2 \subset \sigma$ such that $u_2 > u_1$, $|\zeta_1|, |\zeta_2|$, i.e. there is more than one interval that makes up σ , and ζ_1, ζ_2 are the first and second such intervals, respectively, with positive mass.

Then we make the following sequence of changes (forming new σ), depending on $\frac{\partial}{\partial u_1} \hat{R}(\sigma)$, $\frac{\partial}{\partial u_1} \hat{R}(\sigma)$:

Subcase 1A, $\frac{\partial}{\partial u_1} \hat{R}(\sigma) > 0$: Increase u_1 until $u_1 = u_2$ (exit Case 1), or $\frac{\partial}{\partial u_1} \hat{R}(\sigma) \leq 0$ (go to Case 1B).

Sub-subcase 1AA, $\frac{\partial}{\partial u_1} \hat{R}(\sigma) < 0$, $u_1 > 0$:

Simultaneously, decrease u_1 .

Sub-subcase 1AB, $\frac{\partial}{\partial u_1} \hat{R}(\sigma) \geq 0$ or $u_1 = 0$:

Hold u_1 fixed.

Subcase 1B, $\frac{\partial}{\partial u_1} \hat{R}(\sigma) \leq 0 \Rightarrow \frac{\partial}{\partial u_1} \hat{R}(\sigma) > 0$: Increase u_1 until $u_1 = u_2$ (exit Case 1), or $\frac{\partial}{\partial u_1} \hat{R}(\sigma) \leq 0$ (which implies $\frac{\partial}{\partial u_1} \hat{R}(\sigma) > 0$, i.e. go to Case 1A).

Each of these changes increases $\hat{R}(\sigma)$, due to the direction of the changes in u_1, u_1 and the signs of the appropriate derivatives. Note that these subcases are mutually-exclusive, and one is true as long as there is more than one disjoint interval, $\exists \zeta_1, \zeta_2 \subset \sigma, u_2 > u_1$. Further, note that u_1 is increasing in Subcase 1A and constant in Subcase 1B. Thus, with u_2 fixed and bounded, eventually:

52

[form-feed in source extraction]

$u_1 \rightarrow u_1$, in Subcase 1B (i.e. the first interval collapses to mass 0). OR

$u_1 \rightarrow u_2$, in Subcase 1A (i.e. the first interval merges with the second).

Thus, this sequence of changes increases the reward, and results in there being one fewer interval than before (after combining the bottom 2 intervals by adding the point $u_1 = u_2$ of 0 measure).

Case 1 can be iteratively applied until there is just a single interval $\sigma = (u^*, \infty)$.

That the changes can increase the reward for any other σ implies that such σ cannot be optimal.

Case 2: $\sigma = (u^*, \infty)$, i.e. there is a single interval that makes up σ

By supposition, $\hat{R}(\sigma_0) > \hat{R}(\emptyset)$ and so $\hat{R}((u^*, \infty)) > \hat{R}(\emptyset)$. Further $\hat{R}((u^*, \infty))$ is a continuous function in u^* . Thus, there exists L such that $\forall u^* > L, \hat{R}((u^*, \infty)) > \hat{R}((u^*, \infty))$.

Thus, there exists $u^* \in [0, L]$ such that $\hat{R}((u^*, \infty)) \geq \hat{R}((u^*, \infty))$, $\forall u^* \in \mathbb{R}^+ \cup \{\infty\}$ (continuous functions in a compact domain have a maximum).

Setting where $r(u, \sigma)$ is strictly decreasing.

The proof is extremely similar to the strictly increasing case, with two differences.

First we now need to [U+0001 control character] modify the starting σ_0 so it does not contain an interval (B, ∞) : set $\sigma_0 = (0, L) \cup u_K$

$k=1 (u_k, u_{k+1}) \setminus (B, \infty)$.

Second, each case from above is duplicated but move the policy in the opposite direction to increase the reward. We omit the details of this case for brevity.

Setting where $r(u, \sigma)$ is strictly quasi-convex.

We show that there exists a $\sigma^* = (0, \cdot^*) \cup (u^*, \infty)$, for some $u^*, \cdot^* \in \mathbb{R}^+$, such that $\hat{R}(\sigma^*) \leq \hat{R}(\sigma)$, $\forall \sigma$.

The key is noting that quasi-convexity of $r(u, \sigma)$ in u implies that any σ with three intervals $\zeta_1, \zeta_2, \zeta_3$ can be improved by eliminating the middle interval (or joining it with one of the others).
Case 1: $\exists$ disjoint $\zeta_1 = (0, u_1)$, $\zeta_2 = (\cdot^2, u_2)$, $\zeta_3 = (\cdot^3, u_3)$, s.t. $|\zeta_1|, |\zeta_2|, |\zeta_3| > 0$, i.e. σ is composed of at least three intervals, and $\zeta_1, \zeta_2, \zeta_3$ are the first three such intervals with positive mass. (u_3 may be ∞).

By supposition, $r(u_k, \sigma)$, is strictly quasi-convex in u , and so $r_L(\cdot^k, \sigma)$, is strictly quasi-concave in $\cdot$.

Then, we have:

$\frac{\partial}{\partial \cdot^2} \hat{R}(\sigma) \leq 0$ and
 $\frac{\partial}{\partial \cdot^2} \hat{R}(\sigma) \geq 0 \implies$
 $\frac{\partial}{\partial \cdot^2} \hat{R}(\sigma) > 0$ and
 $\frac{\partial}{\partial \cdot^2} \hat{R}(\sigma) < 0$

$\frac{\partial}{\partial u_1} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) \leq 0$ or
 $\frac{\partial}{\partial u_1} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) \geq 0 \implies$
 $\frac{\partial}{\partial u_1} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) > 0$ or
 $\frac{\partial}{\partial u_1} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) < 0$

$\frac{\partial}{\partial u_2} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) \leq 0$ or
 $\frac{\partial}{\partial u_2} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) \geq 0 \implies$
 $\frac{\partial}{\partial u_2} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) > 0$ or
 $\frac{\partial}{\partial u_2} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) < 0$

Then we make the following sequence of changes (forming new σ):
Subcase 1A, $\frac{\partial}{\partial u_1} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) \leq 0$ and $\frac{\partial}{\partial u_2} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) \geq 0 \implies \frac{\partial}{\partial \cdot^2} \hat{R}(\sigma) > 0$ and $\frac{\partial}{\partial u_2} \frac{\partial}{\partial \cdot^2} \hat{R}(\sigma) < 0$: Increase $\cdot^2$ and decrease u_2 simultaneously until $\cdot^2 = u_2$ (exit Case 1), or $\frac{\partial}{\partial \cdot^2} \hat{R}(\sigma) \geq 0$, or $\frac{\partial}{\partial \cdot^2} \hat{R}(\sigma) \leq 0$ (go to 1B or 1C).

Subcase 1B, $\frac{\partial}{\partial u_1} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) > 0$: Increase u_1 until $u_1 = \cdot^2$ (exit Case 1), or $\frac{\partial}{\partial u_1} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) \leq 0$ (go to 1A or 1C)

Subcase 1C, $\frac{\partial}{\partial u_2} \frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) < 0$: Decrease $\cdot^3$ until $u_2 = \cdot^3$ (exit Case 1), or $\frac{\partial}{\partial \cdot^3} \hat{R}(\sigma) \geq 0$ (go to 1B or 1A).

53

[form-feed in source extraction]

Each of these changes strictly increase $\hat{R}(\sigma)$. 1B and 1C may both be true, in which case we arbitrarily decide between them. At least one of the three subcases is true as long as the Case 1 condition holds. Thus, eventually:

- $\cdot^2 = u_2$, in Subcase 1A (i.e. the middle interval collapses to mass 0). OR
- $u_1 = \cdot^2$, in Subcase 1B (i.e. the first interval merges with the second). OR
- $u_2 = \cdot^3$, in Subcase 1C (i.e. the third interval merges with the second).

Thus, this sequence of changes cannot decrease the reward, and results in there being one fewer interval than before. Case 1 can be iteratively applied until there are just two intervals $\sigma = (0, t_1) \cup (t_2, \infty)$.

That the changes can increase the reward for any other σ implies that such σ cannot be optimal.
Case 2: $\sigma = (0, t_1) \cup (t_2, \infty)$. We need to show that there exists a maxima (t^*_1, t^*_2) . Specifically, we need to eliminate the possible cases where t_1 or t_2 increase to infinity, but the asymptotic values at ∞ produce lower rewards, which would have implied that the maximum is not achieved.

By supposition, $\hat{R}(\sigma) > \hat{R}(\emptyset)$ and so $\hat{R}((0, t_1) \cup (t_2, \infty)) > \hat{R}(\emptyset)$ for $t_1 > 0$ or $t_2 < \infty$.

Further $\hat{R}((0, t_1) \cup (t_2, \infty))$ is a continuous function in t_1, t_2 . Thus $\hat{R}((0, t_1) \cup (t_2, \infty)) \rightarrow \hat{R}(\emptyset)$ as $t_1 \rightarrow 0, t_2 \rightarrow \infty$ together.

Further, $\hat{R}((0, t_1) \cup (t_2, \infty)) \rightarrow \hat{R}((0, \infty))$ as $t_1 \rightarrow \infty$, regardless of how t_2 behaves. Similarly, fixing t_1 , $\hat{R}((0, t_1) \cup (t_2, \infty)) \rightarrow \hat{R}((0, t_1))$ as $t_2 \rightarrow \infty$.

- If $\hat{R}((0, t_1) \cup (t^*_2(t_1), \infty))$ is increasing for $t_1 > T_1$, for however $t^*_2(t_1)$ behaves as a function of t_1 then $\hat{R}((0, \infty)) \geq \hat{R}((0, t_1) \cup (t_2, \infty))$, $\forall t_1 > T_1, t_2$.

- For any fixed t_1 , if $\hat{R}((0, t_1) \cup (t_2, \infty))$ is increasing for $t_2 > T_2$, then $\hat{R}((0, t_1)) \geq \hat{R}((0, t_1) \cup (t_2, \infty))$, $\forall t_2$.

Thus, the maximum is achieved: either

1. $\exists t^*_1 \in (0, \infty) : \hat{R}((0, t^*_1)) \geq \hat{R}((0, t_1) \cup (t_2, \infty))$, $\forall t_1, t_2$
2. $\exists t^*_1, t^*_2 \in [0, \infty) : \hat{R}((0, t^*_1) \cup (t^*_2, \infty)) \geq \hat{R}((0, t_1) \cup (t_2, \infty))$, $\forall t_1, t_2$

Setting where $r(u, \sigma)$ is strictly quasi-concave.

The proof is extremely similar to the strictly quasi-convex case. However, we now need to modify the starting σ_0 so it does not contain an intervals $(0, L)$ or (B, ∞) , and the subsequent modifications also differ directionally.

[U+0001 control character]

Let $\sigma_0 = u_K$

$k=1 (\cdot^k, u_k) \setminus (0, L) \setminus (B, \infty)$. Now, the key step is noting that strict quasi-concavity of $r(u, \sigma)$ implies that any σ with two intervals ζ_1, ζ_2 can be improved by eliminating one (or joining the two).

Case 1: $\exists$ disjoint $\zeta_1 = (\cdot^1, u_1)$, $\zeta_2 = (\cdot^2, u_2)$, s.t. $|\zeta_1|, |\zeta_2| > 0$, i.e. σ is composed of at least two intervals with positive mass, and ζ_1, ζ_2 are the first two such intervals.

By supposition, $r(u_k, \sigma)$, is strictly quasi-concave in u . Then, $r_L(\cdot^k, \sigma)$, is strictly quasi-convex in $\cdot$.

Then, we have:

$\frac{\partial}{\partial \cdot^2}$
 $\frac{\partial}{\partial \cdot^2}$
 $\frac{\partial}{\partial \cdot^2}$

$\hat{R}(\sigma) \leq 0$ and
 $\hat{R}(\sigma) \leq 0 \Rightarrow$
 $\hat{R}(\sigma) > 0$
 ∂^1
 ∂^2
 ∂u_1
 ∂
 ∂
 ∂
 $\hat{R}(\sigma) \leq 0 \Rightarrow$
 $\hat{R}(\sigma) > 0$ or
 $\hat{R}(\sigma) > 0$
 ∂u_1
 ∂^1
 ∂^2
54

[form-feed in source extraction]

Then we make the following sequence of changes (forming new σ):

Subcase 1A, $\partial^1 \hat{R}(\sigma) \leq 0$ and $\partial^2 \hat{R}(\sigma) \leq 0 \Rightarrow \partial u_1 \hat{R}(\sigma) > 0$: Increase u_1 until $u_1 = \partial^2$ (exit Case 1) or $\partial u_1 \hat{R}(\sigma) \leq 0$ (go to 1B or 1C).

Subcase 1B, $\partial^1 \hat{R}(\sigma) > 0$: Increase ∂^1 until $u_1 = \partial^1$ (exit Case 1), or $\partial^1 \hat{R}(\sigma) \leq 0$ (go to 1A or 1C)

Subcase 1C, $\partial^2 \hat{R}(\sigma) > 0$: Increase ∂^2 until $u_2 = \partial^2$ (exit Case 1), or $\partial^2 \hat{R}(\sigma) \leq 0$ (go to 1B or 1A).

Each of these changes strictly increase $R(\sigma)$. 1B and 1C may both be true, in which case arbitrarily decide between them. At least one of the three subcases is true as long as the Case 1 condition holds. Thus, eventually:

- $\partial^2 = u_1$, in Subcase 1A (i.e. the intervals combine). OR
- $u_1 = \partial^1$, in Subcase 1B (i.e. the first interval collapses to mass 0). OR
- $u_2 = \partial^2$, in Subcase 1C (i.e. the second interval collapses to mass 0).

Thus, this sequence of changes cannot decrease the reward, and result in there being one fewer interval than before. Case 1 can be iteratively applied until there is just one interval $\sigma_i = (t_1, t_2)$. That the changes can increase the reward for any other σ implies that such σ cannot be optimal.

Case 2: $\sigma_i = (t_1, t_2)$. As in the same case in the quasi-convex setting. We need to values eliminate the possible cases where t_1 or t_2 increasing to infinity, but the asymptotic values at ∞ produce lower rewards, which would imply that the maximum is not achieved.

Note that $\hat{R}((t_1, t_2))$ is a continuous function in t_1, t_2 . Thus $\hat{R}((t_1, t_2)) \rightarrow \hat{R}(\emptyset)$ as $t_1 \rightarrow t_2$.

Further, $\hat{R}((t_1, t_2)) \rightarrow \hat{R}(\emptyset)$ as $t_1 \rightarrow \infty$, regardless of how $t_2 \geq t_1$ behaves. Similarly, fixing t_1 , $\hat{R}((t_1, t_2)) \rightarrow \hat{R}((t_1, \infty))$ as $t_2 \rightarrow \infty$.

• If $\hat{R}((t_1, t_2))$ is increasing for $t_1 > T_1$, for however $t_2(t_1)$ behaves as a function of t_1 then $\hat{R}(\emptyset) \geq \hat{R}((t_1, t_2))$, $\forall t_1 > T_1, t_2$.

• For any fixed t_1 , if $\hat{R}((t_1, t_2))$ is increasing for $t_2 > T_2$, then $\hat{R}((t_1, \infty)) \geq \hat{R}((t_1, t_2))$, $\forall t_2 > T_2$. Thus, either

1. $\hat{R}(\emptyset) \geq \hat{R}((t_1, t_2))$, $\forall t_1, t_2$
2. $\exists t_1 \in [0, \infty) : \hat{R}((t_1, \infty)) \geq \hat{R}((t_1, t_2))$, $\forall t_1, t_2$
3. $\exists t_1, t_2 \in [0, \infty) : \hat{R}((t_1, t_2)) \geq \hat{R}((t_1, \infty))$, $\forall t_1, t_2$

[Next verbatim source excerpt]

Recall in the dynamic model that we constrain σ_i to be measurable, open, subsets of the $\mathbb{R}^+$. Then, σ_i can be written as a countable union of disjoint subsets of $\mathbb{R}^+$, i.e. $\sigma_i = \cup_{k=0}^{\infty} (\tau_k, \tau_{k+1})$. We further

assume that $\tau_k = \tau_m$, for any k, m ; we can do so without loss of generality by making a measure 0 change to σ_i , by adding $\tau_k = \tau_m$ to σ_i .

Suppose u is an upper-endpoint of σ_i , i.e. $\exists k$ such that $u = \tau_k$. Then, we use ∂u $H(\sigma_i)$ to denote

the derivative of the set function H with respect to u at σ_i . Similarly, $\partial^- H(\sigma_i)$ is the derivative of H at σ_i with respect to a lower-endpoint of σ_i .

We derive ∂u
 $R(w, \{\sigma_1, \sigma_2\})$, ∂^-
 $R(w, \{\sigma_1, \sigma_2\})$. We will make it clear in each instance whether u

refers to the derivative
or ∂^- is an endpoint of σ_1 or σ_2 . For all the derivatives in this subsection ∂u

with respect to an upper endpoint in σ_i , and ∂^- refers to a derivative at a lower endpoint of σ_i . Note that
 $R(w, \{\sigma_1, \sigma_2\}) = -R(w, \{\sigma_1, \sigma_2\})$.

Furthermore:
• We use σ in the function argument when the function depends on policies in both states, and σ_i when it only depends on the policy in state i .
• We use α to denote that “two functions of u have the same sign except where $f(u) = 0$ ”, rather than proportional to.
• All policy equalities are up to measure 0.

Expanded PaperInterface specification

```

V (mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) [MeasureTheory.IsFiniteMeasure mu]
  [mu.InnerRegularCompactLTop] [MeasureTheory.NoAtoms mu] (Rhat : GN21DriverSurgePricing.SingleStateReward)
  (response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ)
  (shape : GN21DriverSurgePricing.Lemma5DerivativeShape) {sigma : GN21DriverSurgePricing.TripPolicy},
  IsOpen sigma →

```



```

sigma ≤ GN21DriverSurgePricing.acceptAllPolicy →
  Rhat 0 < Rhat sigma →
    ∀ {margin : ℝ},
      0 < margin →
        (∀ (tau : GN21DriverSurgePricing.TripPolicy), GN21DriverSurgePricing.GN21SymmDiffContinuousAt mu Rhat tau) →
          (∀ (extra : ℕ),
            ContinuousOn (fun endpoints => Rhat (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints))
              (GN21DriverSurgePricing.gn21Lemma5EndpointDomain shape extra)) →
            (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.positive →
              ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                Rhat sigma - margin ≤ Rhat tau →
                  ∀ (u : GN21DriverSurgePricing.TripLength), 0 < u → 0 < response tau u) →
                (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyIncreasing →
                  ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                    Rhat sigma - margin ≤ Rhat tau → StrictMonoOn (response tau) (Set.Ici 0)) →
                (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyDecreasing →
                  ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                    Rhat sigma - margin ≤ Rhat tau → StrictAntiOn (response tau) (Set.Ici 0)) →
                (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex →
                  ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                    Rhat sigma - margin ≤ Rhat tau →
                      GN21DriverSurgePricing.strictQuasiConvexOnPositive (response tau)) →
                (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave →
                  ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                    Rhat sigma - margin ≤ Rhat tau →
                      ∀ {middleLower middleUpper right : ℝ},
                        0 < middleLower →
                          middleLower < middleUpper →
                            middleUpper < right →
                              response tau middleLower < max (response tau 0) (response tau right)) →
                    (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave →
                      ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                        Rhat sigma - margin ≤ Rhat tau →
                          GN21DriverSurgePricing.strictQuasiConcaveOnPositive (response tau)) →
                    (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave →
                      ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                        Rhat sigma - margin ≤ Rhat tau →
                          ∀ {middle right : ℝ},
                            0 < middle →
                              middle < right →
                                min (response tau 0) (response tau right) < response tau middle) →
                  )
                )
            )
          )
        )
      )
    )
  )
  (H :
    GN21DriverSurgePricing.ProofBridge.GN21Lemma5AEndpointVariation mu Rhat response
      shape sigma margin),
  )
  ∃ policy,
    GN21DriverSurgePricing.lemma5SourcePolicyForm shape policy ∧
      Rhat sigma ≤ Rhat policy ∧
      (¬GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere mu shape sigma →
        Rhat sigma < Rhat policy)

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.lemma5_full_variational_policy_forms

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 5 treats the five stated response-shape cases and concludes a no-worse policy of the corresponding interval form, with strict improvement unless the starting policy already has that form up to null sets. The Spec makes the source's continuous-law, open-policy, equality-modulo-null-sets, and endpoint-derivative conditions explicit rather than replacing them by a name or a policy-form premise.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source-model definitions (continued)

Review row 17

Source locator: cited publication, lines 264-287

Verbatim source input

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq R^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of R^+ . When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is $R(w, \sigma), \liminf_{t \rightarrow \infty}$

$R(w, \sigma, t)$
.
 t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (\emptyset, \infty)$ in the single-state model or $\sigma = \{(\emptyset, \infty), (\emptyset, \infty)\}$ in the dynamic model is optimal with respect to w . In other words, payment function w is incentive compatible if an earnings-maximizing driver (who knows all the primitives, w , and the trip length τ at request time) accepts every trip request.

Expanded PaperInterface specification

```
(∀ (R : GN21DriverSurgePricing.SingleStateReward),  
  ∀ σ ⊆ GN21DriverSurgePricing.acceptAllPolicy, MeasurableSet σ → R σ ≤ R GN21DriverSurgePricing.acceptAllPolicy) ∧  
  (∀ (R : GN21DriverSurgePricing.DynamicReward),  
    GN21DriverSurgePricing.dynamicFeasibleOpenPolicy GN21DriverSurgePricing.acceptAllDynamicPolicy ∧  
    ∀ (σ : Fin 2 → GN21DriverSurgePricing.TripPolicy),  
      GN21DriverSurgePricing.dynamicFeasibleOpenPolicy σ → R σ ≤ R GN21DriverSurgePricing.acceptAllDynamicPolicy)
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_definition_incentive_compatible_realizes_spec

Recorded source-to-Spec screening Verdict: matches

Reason: The one source definition gives both accept-all branches: measurable one-state policies and open measurable dynamic policy pairs. The one transparent Spec is their conjunction, with each source policy domain and accept-all optimality comparison explicit.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims (continued 3)

Review row 18

Source locator: cited publication, lines 264-285

Verbatim source input

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq R^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of R^+ . When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is

$$R(w, \sigma), \liminf_{t \rightarrow \infty}$$

$$R(w, \sigma, t)$$

.

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (\emptyset, \infty)$ in the single-state model or $\sigma = \{(\emptyset, \infty), (\emptyset, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

Theorem 3. Let $R_1 < R_2$ be target earning rates during non-surged and surge states, respectively.

There exist prices $w = \{w_1, w_2\}$ of the form

$$w_i(\tau) = m_i \tau + z_i q_{i \rightarrow j}(\tau),$$

where $m_1, m_2, z_2 \geq 0$ (but z_1 may be either positive or negative), such that the optimal driver policy is to accept every trip in the surge state and all trips up to a certain length in the non-surge state.

R_1

Furthermore, for R

$\in (C, 1)$, there exist fully incentive compatible prices of this form, where

2

$$C = 1 -$$

$$1 - Q_2 (\lambda_{12} T_1 - Q_1) + Q_1 (T_2 \lambda_{1 \rightarrow 2} + Q_2)$$

$$\in [0, 1),$$

$$T_1 Q_2 (\lambda_{1 \rightarrow 2} T_1 - Q_1) + \lambda_{1 \rightarrow 2} (T_2 \lambda_{1 \rightarrow 2} + Q_2)$$

and $T_i = \lambda_i F_i(\sigma_i) T_i((\emptyset, \infty))$, and $Q_i = Q_i((\emptyset, \infty))$. For such prices, the driver policy to accept all requests is the unique optimal driver policy (up to differences of measure 0).

[Next verbatim source excerpt]

$$q_{i \rightarrow j}(\tau) dF_i(\tau)$$

$$Q_i, Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$$

σ_i

z

$$\tau dF_i(\tau)$$

$$T_i, \lambda_i F_i(\sigma_i) T_i(\sigma_i) = 1 + \lambda_i$$

$\tau \in \sigma_i$

z

$$w_i(\tau) dF_i(\tau)$$

$$W_i, \lambda_i F_i(\sigma_i) W_i(\sigma_i) = \lambda_i$$

$\tau \in \sigma_i$

$$\Delta j_i, \Delta(\sigma_j, \sigma_i) = R_j(w_j, \sigma_j) - R_i(w_i, \sigma_i)$$

We assume throughout:

- Distribution of jobs F , F_i is a continuous probability measure, i.e., f, f_i bounded.

- There exists a policy in state 2 that dominates state 1: $\exists \sigma_2$ such that $\Delta(\sigma_2, \sigma_1) > 0, \forall \sigma_1 \subseteq (\emptyset, \infty)$.

- σ, σ_i constrained to be measurable with respect to F, F_i , and σ_i are open.

Expanded PaperInterface specification

```
(∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ)
  (R1 R2 switch12 switch21 : ℝ) [MeasureTheory.NoAtoms (mu 0)] [MeasureTheory.NoAtoms (mu 1)]
  [MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)],
  0 ≤ R1 →
    R1 < R2 →
      0 < arrival 0 →
        0 < arrival 1 →
          0 < switch12 →
            0 < switch21 →
              MeasureTheory.IntegrableOn (fun tau => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
                MeasureTheory.IntegrableOn (fun tau => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
                  GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1 →
                    GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1 →
                      ∃ m z rho,
                        (0 ≤ m 0 ∧ 0 ≤ m 1 ∧ 0 ≤ z 1) ∧
                          GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 0) (arrival 0)
                            (GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 0)
                              GN21DriverSurgePricing.acceptAllPolicy =
                                R1 ∧
```

```

GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 1) (arrival 1)
  (GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 1)
GN21DriverSurgePricing.acceptAllPolicy =
  R2 ∧
  rho 1 = GN21DriverSurgePricing.acceptAllPolicy ∧
  GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoff (rho 0) ∧
  GN21DriverSurgePricing.dynamicOpenOptimal
  (GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival
    switch12 switch21 m z)
  rho) ∧
∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ)
  (R1 R2 switch12 switch21 : ℝ) [MeasureTheory.NoAtoms (mu 0)] [MeasureTheory.NoAtoms (mu 1)]
  [MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)],
0 < R2 →
  GN21DriverSurgePricing.theorem3FeasibilityThresholdC
  (GN21DriverSurgePricing.gn21AcceptAllScaledStateTime (mu 0) (arrival 0))
  (GN21DriverSurgePricing.gn21AcceptAllScaledStateTime (mu 1) (arrival 1))
  (GN21DriverSurgePricing.gn21AcceptAllExitWeightIntegral (mu 0) (arrival 0) switch12 switch21)
  (GN21DriverSurgePricing.gn21AcceptAllExitWeightIntegral (mu 1) (arrival 1) switch21 switch12) switch12 <
  R1 / R2 →
  R1 / R2 < 1 →
  0 < arrival 0 →
  0 < arrival 1 →
  0 < switch12 →
  0 < switch21 →
  MeasureTheory.IntegrableOn (fun tau => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
  MeasureTheory.IntegrableOn (fun tau => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
  GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1 →
  GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1 →
  ∃ m z,
  (0 ≤ m 0 ∧ 0 ≤ m 1 ∧ 0 ≤ z 1) ∧
  GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 0) (arrival 0)
  (GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 0)
  GN21DriverSurgePricing.acceptAllPolicy =
  R1 ∧
  GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 1) (arrival 1)
  (GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 1)
  GN21DriverSurgePricing.acceptAllPolicy =
  R2 ∧
  GN21DriverSurgePricing.dynamicOpenOptimal
  (GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival
    switch12 switch21 m z)
  GN21DriverSurgePricing.acceptAllDynamicPolicy ∧
  ∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
  GN21DriverSurgePricing.dynamicOpenOptimal
  (GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival
    switch12 switch21 m z)
  rho →
  GN21DriverSurgePricing.dynamicAcceptAllAlmostEverywhere mu rho

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem3_structured_pricing

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 3 states both its ordered-target structured-pricing clause and its stricter ratio-interval fully-IC clause under one numbered result. The transparent conjunction preserves the two displayed target domains, structured-price form, target-rate conclusions, policy form, accept-all optimality, and uniqueness up to null sets.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation